

同余 gcd 积性函数筛法

VR+ES+Y:R

目录

1 数论基础

2 欧几里德算法

3 积性函数

4 筛法

整除

- 整除: 设 $a, b \in \mathbb{Z}, a \neq 0$ 。如果 $\exists q \in \mathbb{Z}$, 使得 $b = aq$, 那么就说 b 可被 a 整除, 记作 $a \mid b$, 且称 b 是 a 的倍数, a 是 b 的约数 (因数)。

整除

- 整除: 设 $a, b \in \mathbb{Z}, a \neq 0$ 。如果 $\exists q \in \mathbb{Z}$, 使得 $b = aq$, 那么就说 b 可被 a 整除, 记作 $a \mid b$, 且称 b 是 a 的倍数, a 是 b 的约数 (因数)。
- 整除的性质:

整除

- 整除: 设 $a, b \in \mathbb{Z}, a \neq 0$ 。如果 $\exists q \in \mathbb{Z}$, 使得 $b = aq$, 那么就说 b 可被 a 整除, 记作 $a \mid b$, 且称 b 是 a 的倍数, a 是 b 的约数 (因数)。
- 整除的性质:
- $a \mid b \wedge b \mid c \implies a \mid c$

整除

- 整除: 设 $a, b \in \mathbb{Z}, a \neq 0$ 。如果 $\exists q \in \mathbb{Z}$, 使得 $b = aq$, 那么就说 b 可被 a 整除, 记作 $a \mid b$, 且称 b 是 a 的倍数, a 是 b 的约数 (因数)。
- 整除的性质:
- $a \mid b \wedge b \mid c \implies a \mid c$
- $a \mid b \wedge a \mid c \iff \forall x, y \in \mathbb{Z}, a \mid (xb + yc)$

整除

- 整除: 设 $a, b \in \mathbb{Z}, a \neq 0$ 。如果 $\exists q \in \mathbb{Z}$, 使得 $b = aq$, 那么就说 b 可被 a 整除, 记作 $a \mid b$, 且称 b 是 a 的倍数, a 是 b 的约数 (因数)。
- 整除的性质:
- $a \mid b \wedge b \mid c \implies a \mid c$
- $a \mid b \wedge a \mid c \iff \forall x, y \in \mathbb{Z}, a \mid (xb + yc)$
- 设 $m \neq 0$, 则 $a \mid b \iff ma \mid mb$ 。

整除

- 整除: 设 $a, b \in \mathbb{Z}, a \neq 0$ 。如果 $\exists q \in \mathbb{Z}$, 使得 $b = aq$, 那么就说 b 可被 a 整除, 记作 $a \mid b$, 且称 b 是 a 的倍数, a 是 b 的约数 (因数)。
- 整除的性质:
- $a \mid b \wedge b \mid c \implies a \mid c$
- $a \mid b \wedge a \mid c \iff \forall x, y \in \mathbb{Z}, a \mid (xb + yc)$
- 设 $m \neq 0$, 则 $a \mid b \iff ma \mid mb$ 。
- 设 $a \neq 0, b = qa + c$, 则 $a \mid b \iff a \mid c$ 。

整除

- 整除: 设 $a, b \in \mathbb{Z}, a \neq 0$ 。如果 $\exists q \in \mathbb{Z}$, 使得 $b = aq$, 那么就说 b 可被 a 整除, 记作 $a \mid b$, 且称 b 是 a 的倍数, a 是 b 的约数 (因数)。
- 整除的性质:
- $a \mid b \wedge b \mid c \implies a \mid c$
- $a \mid b \wedge a \mid c \iff \forall x, y \in \mathbb{Z}, a \mid (xb + yc)$
- 设 $m \neq 0$, 则 $a \mid b \iff ma \mid mb$ 。
- 设 $a \neq 0, b = qa + c$, 则 $a \mid b \iff a \mid c$ 。
- 显然约数 (显然因数): 对于整数 $b \neq 0$, ± 1 、 $\pm b$ 是 b 的显然约数。当 $b = \pm 1$ 时, b 只有两个显然约数。

素数

- 素数：设整数 $p \neq 0, \pm 1$ 。如果 p 除了显然约数外没有其他约数，那么称 p 为素数（不可约数）。

素数

- 素数：设整数 $p \neq 0, \pm 1$ 。如果 p 除了显然约数外没有其他约数，那么称 p 为素数（不可约数）。
- 合数：若整数 $p \neq 0, \pm 1$ 且 p 不是素数，则称 p 为合数。

素数

- 素数：设整数 $p \neq 0, \pm 1$ 。如果 p 除了显然约数外没有其他约数，那么称 p 为素数（不可约数）。
- 合数：若整数 $p \neq 0, \pm 1$ 且 p 不是素数，则称 p 为合数。
- p 与 $-p$ 是同为素数或者同为合数。下文中，如果没有特别说明，素数总是指正的素数。

素数

- 素数：设整数 $p \neq 0, \pm 1$ 。如果 p 除了显然约数外没有其他约数，那么称 p 为素数（不可约数）。
- 合数：若整数 $p \neq 0, \pm 1$ 且 p 不是素数，则称 p 为合数。
- p 与 $-p$ 是同为素数或者同为合数。下文中，如果没有特别说明，素数总是指正的素数。
- 对于合数 a ，一定存在素数 $p \leq \sqrt{a}$ 满足 $p \mid a$ 。

素数

- 素数：设整数 $p \neq 0, \pm 1$ 。如果 p 除了显然约数外没有其他约数，那么称 p 为素数（不可约数）。
- 合数：若整数 $p \neq 0, \pm 1$ 且 p 不是素数，则称 p 为合数。
- p 与 $-p$ 是同为素数或者同为合数。下文中，如果没有特别说明，素数总是指正的素数。
- 对于合数 a ，一定存在素数 $p \leq \sqrt{a}$ 满足 $p \mid a$ 。
- 算术基本引理：设 p 是素数， $p \mid ab$ ，那么 $p \mid a$ 和 $p \mid b$ 至少有一个成立。

素数

- 素数：设整数 $p \neq 0, \pm 1$ 。如果 p 除了显然约数外没有其他约数，那么称 p 为素数（不可约数）。
- 合数：若整数 $p \neq 0, \pm 1$ 且 p 不是素数，则称 p 为合数。
- p 与 $-p$ 是同为素数或者同为合数。下文中，如果没有特别说明，素数总是指正的素数。
- 对于合数 a ，一定存在素数 $p \leq \sqrt{a}$ 满足 $p \mid a$ 。
- 算术基本引理：设 p 是素数， $p \mid ab$ ，那么 $p \mid a$ 和 $p \mid b$ 至少有一个成立。
- 算术基本定理：设正整数 a ，那么必有表示： $a = p_1 p_2 \cdots p_s$ 。其中 p_j 是素数。并且在不计次序的意义下，该表示唯一。

素数

- 素数：设整数 $p \neq 0, \pm 1$ 。如果 p 除了显然约数外没有其他约数，那么称 p 为素数（不可约数）。
- 合数：若整数 $p \neq 0, \pm 1$ 且 p 不是素数，则称 p 为合数。
- p 与 $-p$ 是同为素数或者同为合数。下文中，如果没有特别说明，素数总是指正的素数。
- 对于合数 a ，一定存在素数 $p \leq \sqrt{a}$ 满足 $p \mid a$ 。
- 算术基本引理：设 p 是素数， $p \mid ab$ ，那么 $p \mid a$ 和 $p \mid b$ 至少有一个成立。
- 算术基本定理：设正整数 a ，那么必有表示： $a = p_1 p_2 \cdots p_s$ 。其中 p_j 是素数。并且在不计次序的意义下，该表示唯一。
- 标准素因数分解式： $a = p_1^{k_1} p_2^{k_2} \cdots p_s^{k_s}, p_1 < p_2 < \cdots < p_s$ 。

gcd

- 公约数：一组整数的公约数，是指同时是这组数中每一个数的约数的数。 ± 1 是任意一组整数的公约数。

gcd

- 公约数：一组整数的公约数，是指同时是这组数中每一个数的约数的数。 ± 1 是任意一组整数的公约数。
- 最大公约数：一组整数的最大公约数，是指所有公约数里面最大的一个。 $a_1, \dots, a_n$ 的最大公约数记为 $\gcd(a_1, \dots, a_n)$ 。

gcd

- 公约数：一组整数的公约数，是指同时是这组数中每一个数的约数的数。 ± 1 是任意一组整数的公约数。
- 最大公约数：一组整数的最大公约数，是指所有公约数里面最大的一个。 $a_1, \dots, a_n$ 的最大公约数记为 $\gcd(a_1, \dots, a_n)$ 。
- 互素：一组整数的最大公约数为 1，则称这组整数互素（既约）。

gcd

- 公约数：一组整数的公约数，是指同时是这组数中每一个数的约数的数。 ± 1 是任意一组整数的公约数。
- 最大公约数：一组整数的最大公约数，是指所有公约数里面最大的一个。 $a_1, \dots, a_n$ 的最大公约数记为 $\gcd(a_1, \dots, a_n)$ 。
- 互素：一组整数的最大公约数为 1，则称这组整数互素（既约）。
- 公倍数：一组整数的公倍数，是指同时是这组数中每一个数的倍数的数。0 是任意一组整数的公倍数。

gcd

- 公约数：一组整数的公约数，是指同时是这组数中每一个数的约数的数。 ± 1 是任意一组整数的公约数。
- 最大公约数：一组整数的最大公约数，是指所有公约数里面最大的一个。 $a_1, \dots, a_n$ 的最大公约数记为 $\gcd(a_1, \dots, a_n)$ 。
- 互素：一组整数的最大公约数为 1，则称这组整数互素（既约）。
- 公倍数：一组整数的公倍数，是指同时是这组数中每一个数的倍数的数。0 是任意一组整数的公倍数。
- 最小公倍数：一组整数的最小公倍数，是指所有正的公倍数里面最小的一个。 $a_1, \dots, a_n$ 的最小公倍数记为 $\text{lcm}(a_1, \dots, a_n)$ 。

gcd

- 公约数：一组整数的公约数，是指同时是这组数中每一个数的约数的数。 ± 1 是任意一组整数的公约数。
- 最大公约数：一组整数的最大公约数，是指所有公约数里面最大的一个。 $a_1, \dots, a_n$ 的最大公约数记为 $\gcd(a_1, \dots, a_n)$ 。
- 互素：一组整数的最大公约数为 1，则称这组整数互素（既约）。
- 公倍数：一组整数的公倍数，是指同时是这组数中每一个数的倍数的数。0 是任意一组整数的公倍数。
- 最小公倍数：一组整数的最小公倍数，是指所有正的公倍数里面最小的一个。 $a_1, \dots, a_n$ 的最小公倍数记为 $\text{lcm}(a_1, \dots, a_n)$ 。
- 通过观察标准素因数分解式可以发现，最大公约数是将每个素数的次数取 $\min$ ，最小公倍数是将每个素数的次数取 $\max$ 。由于 $a + b = \max(a, b) + \min(a, b)$ ，所以有 $ab = \gcd(a, b) \text{lcm}(a, b)$ 。

gcd

- 公约数：一组整数的公约数，是指同时是这组数中每一个数的约数的数。 ± 1 是任意一组整数的公约数。
- 最大公约数：一组整数的最大公约数，是指所有公约数里面最大的一个。 $a_1, \dots, a_n$ 的最大公约数记为 $\gcd(a_1, \dots, a_n)$ 。
- 互素：一组整数的最大公约数为 1，则称这组整数互素（既约）。
- 公倍数：一组整数的公倍数，是指同时是这组数中每一个数的倍数的数。0 是任意一组整数的公倍数。
- 最小公倍数：一组整数的最小公倍数，是指所有正的公倍数里面最小的一个。 $a_1, \dots, a_n$ 的最小公倍数记为 $\text{lcm}(a_1, \dots, a_n)$ 。
- 通过观察标准素因数分解式可以发现，最大公约数是将每个素数的次数取 $\min$ ，最小公倍数是将每个素数的次数取 $\max$ 。由于 $a + b = \max(a, b) + \min(a, b)$ ，所以有 $ab = \gcd(a, b) \text{lcm}(a, b)$ 。
- 公约数一定是最大公约数的约数，公倍数一定是最小公倍数的倍数，反之亦然。

同余

- 同余: 设整数 $m \neq 0$ 。若 $m \mid (a - b)$, 称 m 为模数 (模), a 同余于 b 模 m , b 是 a 对模 m 的剩余。记作 $a \equiv b \pmod{m}$ 。

同余

- 同余：设整数 $m \neq 0$ 。若 $m \mid (a - b)$ ，称 m 为模数（模）， a 同余于 b 模 m ， b 是 a 对模 m 的剩余。记作 $a \equiv b \pmod{m}$ 。
- 根据整除的性质，上述同余式也等价于 $a \equiv b \pmod{-m}$ 。下文如果没有特别说明，模数总是正整数。

同余

- 同余：设整数 $m \neq 0$ 。若 $m \mid (a - b)$ ，称 m 为模数（模）， a 同余于 b 模 m ， b 是 a 对模 m 的剩余。记作 $a \equiv b \pmod{m}$ 。
- 根据整除的性质，上述同余式也等价于 $a \equiv b \pmod{-m}$ 。下文如果没有特别说明，模数总是正整数。
- 性质：

同余

- 同余：设整数 $m \neq 0$ 。若 $m \mid (a - b)$ ，称 m 为模数（模）， a 同余于 b 模 m ， b 是 a 对模 m 的剩余。记作 $a \equiv b \pmod{m}$ 。
- 根据整除的性质，上述同余式也等价于 $a \equiv b \pmod{-m}$ 。下文如果没有特别说明，模数总是正整数。
- 性质：
- 自反性： $a \equiv a \pmod{m}$ 。

同余

- 同余：设整数 $m \neq 0$ 。若 $m \mid (a - b)$ ，称 m 为模数（模）， a 同余于 b 模 m ， b 是 a 对模 m 的剩余。记作 $a \equiv b \pmod{m}$ 。
- 根据整除的性质，上述同余式也等价于 $a \equiv b \pmod{-m}$ 。下文如果没有特别说明，模数总是正整数。
- 性质：
- 自反性： $a \equiv a \pmod{m}$ 。
- 对称性： $a \equiv b \pmod{m} \iff b \equiv a \pmod{m}$ 。

同余

- 同余：设整数 $m \neq 0$ 。若 $m \mid (a - b)$ ，称 m 为模数（模）， a 同余于 b 模 m ， b 是 a 对模 m 的剩余。记作 $a \equiv b \pmod{m}$ 。
- 根据整除的性质，上述同余式也等价于 $a \equiv b \pmod{-m}$ 。下文如果没有特别说明，模数总是正整数。
- 性质：
- 自反性： $a \equiv a \pmod{m}$ 。
- 对称性： $a \equiv b \pmod{m} \iff b \equiv a \pmod{m}$ 。
- 传递性： $a \equiv b \pmod{m} \wedge b \equiv c \pmod{m} \implies a \equiv c \pmod{m}$ 。

同余

- 同余：设整数 $m \neq 0$ 。若 $m \mid (a - b)$ ，称 m 为模数（模）， a 同余于 b 模 m ， b 是 a 对模 m 的剩余。记作 $a \equiv b \pmod{m}$ 。
- 根据整除的性质，上述同余式也等价于 $a \equiv b \pmod{-m}$ 。下文如果没有特别说明，模数总是正整数。
- 性质：
- 自反性： $a \equiv a \pmod{m}$ 。
- 对称性： $a \equiv b \pmod{m} \iff b \equiv a \pmod{m}$ 。
- 传递性： $a \equiv b \pmod{m} \wedge b \equiv c \pmod{m} \implies a \equiv c \pmod{m}$ 。
- 线性运算：设 $a \equiv b \pmod{m}, c \equiv d \pmod{m}$ 则 $a \pm c \equiv b \pm d \pmod{m}, a \times c \equiv b \times d \pmod{m}$ 。

同余

- 同余：设整数 $m \neq 0$ 。若 $m \mid (a - b)$ ，称 m 为模数（模）， a 同余于 b 模 m ， b 是 a 对模 m 的剩余。记作 $a \equiv b \pmod{m}$ 。
- 根据整除的性质，上述同余式也等价于 $a \equiv b \pmod{-m}$ 。下文如果没有特别说明，模数总是正整数。
- 性质：
- 自反性： $a \equiv a \pmod{m}$ 。
- 对称性： $a \equiv b \pmod{m} \iff b \equiv a \pmod{m}$ 。
- 传递性： $a \equiv b \pmod{m} \wedge b \equiv c \pmod{m} \implies a \equiv c \pmod{m}$ 。
- 线性运算：设 $a \equiv b \pmod{m}, c \equiv d \pmod{m}$ 则 $a \pm c \equiv b \pm d \pmod{m}, a \times c \equiv b \times d \pmod{m}$ 。
- 设 $k \neq 0$ 则 $a \equiv b \pmod{m} \iff ak \equiv bk \pmod{mk}$ 。

同余

- 若 $d \mid m, a \equiv b \pmod{m}$ 则 $a \equiv b \pmod{d}$ 。

同余

- 若 $d \mid m, a \equiv b \pmod{m}$ 则 $a \equiv b \pmod{d}$ 。
- $a \equiv b \pmod{m} \implies \gcd(a, m) = \gcd(b, m)$ 。

同余

- 若 $d \mid m, a \equiv b \pmod{m}$ 则 $a \equiv b \pmod{d}$ 。
- $a \equiv b \pmod{m} \implies \gcd(a, m) = \gcd(b, m)$ 。
- 线性同余方程：形如 $ax \equiv c \pmod{b}$ 的方程被称为线性同余方程。该方程与 $ax + by = c$ 等价。

同余

- 若 $d \mid m, a \equiv b \pmod{m}$ 则 $a \equiv b \pmod{d}$ 。
- $a \equiv b \pmod{m} \implies \gcd(a, m) = \gcd(b, m)$ 。
- 线性同余方程：形如 $ax \equiv c \pmod{b}$ 的方程被称为线性同余方程。该方程与 $ax + by = c$ 等价。
- 裴蜀定理：设 a, b 是不全为零的整数，则存在整数 x, y ，使得 $ax + by = \gcd(a, b)$ 。

同余

- 若 $d \mid m, a \equiv b \pmod{m}$ 则 $a \equiv b \pmod{d}$ 。
- $a \equiv b \pmod{m} \implies \gcd(a, m) = \gcd(b, m)$ 。
- 线性同余方程：形如 $ax \equiv c \pmod{b}$ 的方程被称为线性同余方程。该方程与 $ax + by = c$ 等价。
- 裴蜀定理：设 a, b 是不全为零的整数，则存在整数 x, y ，使得 $ax + by = \gcd(a, b)$ 。
- 用扩展欧几里德算法可以构造性证明。因此 $ax \equiv c \pmod{b}$ 有解的充要条件为 $c \mid \gcd(a, b)$ 。

同余

- 若 $d \mid m, a \equiv b \pmod{m}$ 则 $a \equiv b \pmod{d}$ 。
- $a \equiv b \pmod{m} \implies \gcd(a, m) = \gcd(b, m)$ 。
- 线性同余方程：形如 $ax \equiv c \pmod{b}$ 的方程被称为线性同余方程。该方程与 $ax + by = c$ 等价。
- 裴蜀定理：设 a, b 是不全为零的整数，则存在整数 x, y ，使得 $ax + by = \gcd(a, b)$ 。
- 用扩展欧几里德算法可以构造性证明。因此 $ax \equiv c \pmod{b}$ 有解的充要条件为 $c \mid \gcd(a, b)$ 。
- 乘法逆元：若 x 满足 $ax \equiv 1 \pmod{m}$ ，则称 x 为 $a \bmod b$ 的逆元，记作 a^{-1} 。

同余

- 若 $d \mid m, a \equiv b \pmod{m}$ 则 $a \equiv b \pmod{d}$ 。
- $a \equiv b \pmod{m} \implies \gcd(a, m) = \gcd(b, m)$ 。
- 线性同余方程：形如 $ax \equiv c \pmod{b}$ 的方程被称为线性同余方程。该方程与 $ax + by = c$ 等价。
- 裴蜀定理：设 a, b 是不全为零的整数，则存在整数 x, y ，使得 $ax + by = \gcd(a, b)$ 。
- 用扩展欧几里德算法可以构造性证明。因此 $ax \equiv c \pmod{b}$ 有解的充要条件为 $c \mid \gcd(a, b)$ 。
- 乘法逆元：若 x 满足 $ax \equiv 1 \pmod{m}$ ，则称 x 为 $a \bmod b$ 的逆元，记作 a^{-1} 。
- 当 a, b 互素时 $a \bmod b$ 的逆元在模意义下存在且唯一。

目录

1 数论基础

2 欧几里德算法

- 扩展欧几里德算法
- 类欧几里德算法
- 万能欧几里德算法

3 积性函数

4 筛法

欧几里德算法

- 要解决的问题：给出两个非负整数 a, b 求出 $\gcd(a, b)$ 。

欧几里德算法

- 要解决的问题：给出两个非负整数 a, b 求出 $\gcd(a, b)$ 。
- 结论：对于 $b \neq 0$ ，有 $\gcd(a, b) = \gcd(b, a \bmod b)$ 。

欧几里德算法

- 要解决的问题：给出两个非负整数 a, b 求出 $\gcd(a, b)$ 。
- 结论：对于 $b \neq 0$ ，有 $\gcd(a, b) = \gcd(b, a \bmod b)$ 。
- 证明： $a \equiv a \bmod b \pmod{b}$ ，所以
 $\gcd(a, b) = \gcd(a \bmod b, b) = \gcd(b, a \bmod b)$ 。

欧几里德算法

- 要解决的问题：给出两个非负整数 a, b 求出 $\gcd(a, b)$ 。
- 结论：对于 $b \neq 0$ ，有 $\gcd(a, b) = \gcd(b, a \bmod b)$ 。
- 证明： $a \equiv a \bmod b \pmod{b}$ ，所以
 $\gcd(a, b) = \gcd(a \bmod b, b) = \gcd(b, a \bmod b)$ 。
- 解法：若 $b = 0$ 则根据定义 $\gcd(a, b) = a$ ，否则递归计算 $\gcd(b, a \bmod b)$ 。

欧几里德算法

- 要解决的问题：给出两个非负整数 a, b 求出 $\gcd(a, b)$ 。
- 结论：对于 $b \neq 0$ ，有 $\gcd(a, b) = \gcd(b, a \bmod b)$ 。
- 证明： $a \equiv a \bmod b \pmod{b}$ ，所以
 $\gcd(a, b) = \gcd(a \bmod b, b) = \gcd(b, a \bmod b)$ 。
- 解法：若 $b = 0$ 则根据定义 $\gcd(a, b) = a$ ，否则递归计算 $\gcd(b, a \bmod b)$ 。
- 复杂度：由于 $a \bmod b < b$ ，所以除了递归的最外层之外，每一层都有 $a > b$ ，而 $a - a \bmod b = b \times \lfloor \frac{a}{b} \rfloor \geq b > a \bmod b$ ，即 $2(a \bmod b) < a$ 。

欧几里德算法

- 要解决的问题：给出两个非负整数 a, b 求出 $\gcd(a, b)$ 。
- 结论：对于 $b \neq 0$ ，有 $\gcd(a, b) = \gcd(b, a \bmod b)$ 。
- 证明： $a \equiv a \bmod b \pmod{b}$ ，所以
 $\gcd(a, b) = \gcd(a \bmod b, b) = \gcd(b, a \bmod b)$ 。
- 解法：若 $b = 0$ 则根据定义 $\gcd(a, b) = a$ ，否则递归计算 $\gcd(b, a \bmod b)$ 。
- 复杂度：由于 $a \bmod b < b$ ，所以除了递归的最外层之外，每一层都有 $a > b$ ，而 $a - a \bmod b = b \times \lfloor \frac{a}{b} \rfloor \geq b > a \bmod b$ ，即 $2(a \bmod b) < a$ 。
- 因此每两层递归 a 的值至少减少一半，时间复杂度为 $\mathcal{O}(\log a)$ 。

扩展欧几里德算法

- 要解决的问题：给出两个整数 a, b 求出 $ax + by = \gcd(a, b)$ 的一组整数解。

扩展欧几里德算法

- 要解决的问题：给出两个整数 a, b 求出 $ax + by = \gcd(a, b)$ 的一组整数解。
- 结论：对于 $b \neq 0$ ，设已经求出 x_2, y_2 使得 $bx_2 + (a \bmod b)y_2 = \gcd(b, a \bmod b)$ ，则原方程的一组解为 $x_1 = y_2, y_1 = x_2 - \lfloor \frac{a}{b} \rfloor y_2$ 。

扩展欧几里德算法

- 要解决的问题：给出两个整数 a, b 求出 $ax + by = \gcd(a, b)$ 的一组整数解。
- 结论：对于 $b \neq 0$ ，设已经求出 x_2, y_2 使得 $bx_2 + (a \bmod b)y_2 = \gcd(b, a \bmod b)$ ，则原方程的一组解为 $x_1 = y_2, y_1 = x_2 - \lfloor \frac{a}{b} \rfloor y_2$ 。
- 推导： $bx_2 + (a \bmod b)y_2 = \gcd(b, a \bmod b)$

扩展欧几里德算法

- 要解决的问题：给出两个整数 a, b 求出 $ax + by = \gcd(a, b)$ 的一组整数解。
- 结论：对于 $b \neq 0$ ，设已经求出 x_2, y_2 使得 $bx_2 + (a \bmod b)y_2 = \gcd(b, a \bmod b)$ ，则原方程的一组解为 $x_1 = y_2, y_1 = x_2 - \lfloor \frac{a}{b} \rfloor y_2$ 。
- 推导： $bx_2 + (a \bmod b)y_2 = \gcd(b, a \bmod b)$
- $bx_2 + (a - b\lfloor \frac{a}{b} \rfloor)y_2 = \gcd(a, b)$

扩展欧几里德算法

- 要解决的问题：给出两个整数 a, b 求出 $ax + by = \gcd(a, b)$ 的一组整数解。
- 结论：对于 $b \neq 0$ ，设已经求出 x_2, y_2 使得 $bx_2 + (a \bmod b)y_2 = \gcd(b, a \bmod b)$ ，则原方程的一组解为 $x_1 = y_2, y_1 = x_2 - \lfloor \frac{a}{b} \rfloor y_2$ 。
- 推导： $bx_2 + (a \bmod b)y_2 = \gcd(b, a \bmod b)$
- $bx_2 + (a - b\lfloor \frac{a}{b} \rfloor)y_2 = \gcd(a, b)$
- $ay_2 + bx_2 - b\lfloor \frac{a}{b} \rfloor y_2 = \gcd(a, b)$

扩展欧几里德算法

- 要解决的问题：给出两个整数 a, b 求出 $ax + by = \gcd(a, b)$ 的一组整数解。
- 结论：对于 $b \neq 0$ ，设已经求出 x_2, y_2 使得 $bx_2 + (a \bmod b)y_2 = \gcd(b, a \bmod b)$ ，则原方程的一组解为 $x_1 = y_2, y_1 = x_2 - \lfloor \frac{a}{b} \rfloor y_2$ 。
- 推导： $bx_2 + (a \bmod b)y_2 = \gcd(b, a \bmod b)$
- $bx_2 + (a - b\lfloor \frac{a}{b} \rfloor)y_2 = \gcd(a, b)$
- $ay_2 + bx_2 - b\lfloor \frac{a}{b} \rfloor y_2 = \gcd(a, b)$
- $ay_2 + b(x_2 - \lfloor \frac{a}{b} \rfloor y_2) = \gcd(a, b)$

扩展欧几里德算法

- 要解决的问题：给出两个整数 a, b 求出 $ax + by = \gcd(a, b)$ 的一组整数解。
- 结论：对于 $b \neq 0$ ，设已经求出 x_2, y_2 使得 $bx_2 + (a \bmod b)y_2 = \gcd(b, a \bmod b)$ ，则原方程的一组解为 $x_1 = y_2, y_1 = x_2 - \lfloor \frac{a}{b} \rfloor y_2$ 。
- 推导： $bx_2 + (a \bmod b)y_2 = \gcd(b, a \bmod b)$
- $bx_2 + (a - b\lfloor \frac{a}{b} \rfloor)y_2 = \gcd(a, b)$
- $ay_2 + bx_2 - b\lfloor \frac{a}{b} \rfloor y_2 = \gcd(a, b)$
- $ay_2 + b(x_2 - \lfloor \frac{a}{b} \rfloor y_2) = \gcd(a, b)$
- 解法：若 $b = 0$ 则直接求出一组解 $x = 1, y = 0$ ，否则递归计算 $a' = b, b' = a \bmod b$ 的解，再用结论计算原问题的解。

扩展欧几里德算法

- 要解决的问题：给出两个整数 a, b 求出 $ax + by = \gcd(a, b)$ 的一组整数解。
- 结论：对于 $b \neq 0$ ，设已经求出 x_2, y_2 使得 $bx_2 + (a \bmod b)y_2 = \gcd(b, a \bmod b)$ ，则原方程的一组解为 $x_1 = y_2, y_1 = x_2 - \lfloor \frac{a}{b} \rfloor y_2$ 。
- 推导： $bx_2 + (a \bmod b)y_2 = \gcd(b, a \bmod b)$
- $bx_2 + (a - b\lfloor \frac{a}{b} \rfloor)y_2 = \gcd(a, b)$
- $ay_2 + bx_2 - b\lfloor \frac{a}{b} \rfloor y_2 = \gcd(a, b)$
- $ay_2 + b(x_2 - \lfloor \frac{a}{b} \rfloor y_2) = \gcd(a, b)$
- 解法：若 $b = 0$ 则直接求出一组解 $x = 1, y = 0$ ，否则递归计算 $a' = b, b' = a \bmod b$ 的解，再用结论计算原问题的解。
- 用归纳法可证明求解过程中 $|x| \leq |b|, |y| \leq |a|$ ，时间复杂度同欧几里德算法。

扩展欧几里德算法

- 另外还可以用矩阵来解释迭代版本的扩展欧几里德算法：欧几里德算法的过程可以描述为

$$\begin{bmatrix} x' & \gcd(a, b) \\ y' & 0 \end{bmatrix} = \cdots \begin{bmatrix} 0 & 1 \\ 1 & -\lfloor \frac{a}{b} \rfloor \end{bmatrix} \begin{bmatrix} 1 & a \\ 0 & b \end{bmatrix}$$

扩展欧几里德算法

- 另外还可以用矩阵来解释迭代版本的扩展欧几里德算法：欧几里德算法的过程可以描述为

$$\begin{bmatrix} x' & \gcd(a, b) \\ y' & 0 \end{bmatrix} = \cdots \begin{bmatrix} 0 & 1 \\ 1 & -\lfloor \frac{a}{b} \rfloor \end{bmatrix} \begin{bmatrix} 1 & a \\ 0 & b \end{bmatrix}$$

- 设

$$\begin{bmatrix} x' & \gcd(a, b) \\ y' & 0 \end{bmatrix} = \begin{bmatrix} x_1 & x_2 \\ x_3 & x_4 \end{bmatrix} \begin{bmatrix} 1 & a \\ 0 & b \end{bmatrix}$$

有 $ax_1 + bx_2 = \gcd(a, b)$ ，并且 $x' = x_1$ ，所以迭代版本的扩展欧几里德算法本质上是在维护那个矩阵。

扩展欧几里德算法

- 另外还可以用矩阵来解释迭代版本的扩展欧几里德算法：欧几里德算法的过程可以描述为

$$\begin{bmatrix} x' & \gcd(a, b) \\ y' & 0 \end{bmatrix} = \cdots \begin{bmatrix} 0 & 1 \\ 1 & -\lfloor \frac{a}{b} \rfloor \end{bmatrix} \begin{bmatrix} 1 & a \\ 0 & b \end{bmatrix}$$

- 设

$$\begin{bmatrix} x' & \gcd(a, b) \\ y' & 0 \end{bmatrix} = \begin{bmatrix} x_1 & x_2 \\ x_3 & x_4 \end{bmatrix} \begin{bmatrix} 1 & a \\ 0 & b \end{bmatrix}$$

有 $ax_1 + bx_2 = \gcd(a, b)$ ，并且 $x' = x_1$ ，所以迭代版本的扩展欧几里德算法本质上是在维护那个矩阵。

- 原问题通解：考虑等价的同余方程 $ax \equiv \gcd(a, b) \pmod{b}$ ，同除 $g = \gcd(a, b)$ 可得 $a'x \equiv 1 \pmod{b'}$ ，其中 $a' = \frac{a}{g}, b' = \frac{b}{g}$ 。

扩展欧几里德算法

- 另外还可以用矩阵来解释迭代版本的扩展欧几里德算法：欧几里德算法的过程可以描述为

$$\begin{bmatrix} x' & \gcd(a, b) \\ y' & 0 \end{bmatrix} = \cdots \begin{bmatrix} 0 & 1 \\ 1 & -\lfloor \frac{a}{b} \rfloor \end{bmatrix} \begin{bmatrix} 1 & a \\ 0 & b \end{bmatrix}$$

- 设

$$\begin{bmatrix} x' & \gcd(a, b) \\ y' & 0 \end{bmatrix} = \begin{bmatrix} x_1 & x_2 \\ x_3 & x_4 \end{bmatrix} \begin{bmatrix} 1 & a \\ 0 & b \end{bmatrix}$$

有 $ax_1 + bx_2 = \gcd(a, b)$ ，并且 $x' = x_1$ ，所以迭代版本的扩展欧几里德算法本质上是在维护那个矩阵。

- 原问题通解：考虑等价的同余方程 $ax \equiv \gcd(a, b) \pmod{b}$ ，同除 $g = \gcd(a, b)$ 可得 $a'x \equiv 1 \pmod{b'}$ ，其中 $a' = \frac{a}{g}, b' = \frac{b}{g}$ 。
- 设 $x = x_0, y = y_0$ 是刚才求出的一个解，由乘法逆元的唯一性， $x \equiv x_0 \pmod{b'}$ ，因此 $x = x_0 + Cb', y = y_0 - Ca'$ 即为原问题的通解。

类欧几里德算法

- 要解决的问题：给出非负整数 $a, b, c, n (c \neq 0)$ 求出

$$f(a, b, c, n) = \sum_{i=0}^n \left\lfloor \frac{ai + b}{c} \right\rfloor$$

类欧几里德算法

- 要解决的问题：给出非负整数 $a, b, c, n (c \neq 0)$ 求出

$$f(a, b, c, n) = \sum_{i=0}^n \lfloor \frac{ai + b}{c} \rfloor$$

- 首先，如果 $a \geq c$ 或 $b \geq c$ 则：

$$\begin{aligned} f(a, b, c, n) &= \sum_{i=0}^n \lfloor \frac{ai + b}{c} \rfloor \\ &= \sum_{i=0}^n \lfloor \frac{(a \bmod c)i + (a - a \bmod c)i + (b \bmod c) + (b - b \bmod c)}{c} \rfloor \\ &= \sum_{i=0}^n \lfloor \frac{(a \bmod c)i + (b \bmod c)}{c} \rfloor + \lfloor \frac{a}{c} \rfloor i + \lfloor \frac{b}{c} \rfloor \\ &= \frac{n(n+1)}{2} \lfloor \frac{a}{c} \rfloor + (n+1) \lfloor \frac{b}{c} \rfloor + f(a \bmod c, b \bmod c, c, n) \end{aligned}$$

类欧几里德算法

- 接下来保证了 $0 \leq a, b < c$, 设 $m = \lfloor \frac{an+b}{c} \rfloor$ 。若 $m = 0$ (包含了 $a = 0$ 的情况) 则 $f(a, b, c, n) = 0$, 否则

$$f(a, b, c, n) = \sum_{i=0}^n \lfloor \frac{ai+b}{c} \rfloor = \sum_{i=0}^n \sum_{j=0}^{m-1} \left[j \leq \lfloor \frac{ai+b}{c} \rfloor - 1 \right]$$

$$j \leq \lfloor \frac{ai+b}{c} \rfloor - 1 \iff j+1 \leq \frac{ai+b}{c} \iff cj + c - b \leq ai$$

$$\iff cj + c - b - 1 < ai \iff \lfloor \frac{cj + c - b - 1}{a} \rfloor < i$$

类欧几里德算法

- 接下来保证了 $0 \leq a, b < c$, 设 $m = \lfloor \frac{an+b}{c} \rfloor$ 。若 $m = 0$ (包含了 $a = 0$ 的情况) 则 $f(a, b, c, n) = 0$, 否则

$$f(a, b, c, n) = \sum_{i=0}^n \lfloor \frac{ai+b}{c} \rfloor = \sum_{i=0}^n \sum_{j=0}^{m-1} \left[j \leq \lfloor \frac{ai+b}{c} \rfloor - 1 \right]$$

$$j \leq \lfloor \frac{ai+b}{c} \rfloor - 1 \iff j+1 \leq \frac{ai+b}{c} \iff cj + c - b \leq ai$$

$$\iff cj + c - b - 1 < ai \iff \lfloor \frac{cj + c - b - 1}{a} \rfloor < i$$

- 考虑 $\lfloor \frac{cj+c-b-1}{a} \rfloor$ 的取值范围, 发现

$$\begin{aligned} 0 \leq j \leq \frac{an+b}{c} - 1 &\implies \frac{c-b-1}{a} \leq \frac{cj+c-b-1}{a} \leq \frac{an-1}{a} \\ &\implies 0 \leq \lfloor \frac{cj+c-b-1}{a} \rfloor < n \end{aligned}$$

类欧几里德算法

■ 所以可以推出:

$$\begin{aligned} f(a, b, c, n) &= \sum_{j=0}^{m-1} \sum_{i=0}^n \left[\left\lfloor \frac{cj + c - b - 1}{a} \right\rfloor < i \right] \\ &= \sum_{j=0}^{m-1} \sum_{i=1}^n \left[\left\lfloor \frac{cj + c - b - 1}{a} \right\rfloor < i \right] \\ &= \sum_{j=0}^{m-1} \left(n - \sum_{i=1}^n \left[i \leq \left\lfloor \frac{cj + c - b - 1}{a} \right\rfloor \right] \right) \\ &= nm - \sum_{j=0}^{m-1} \left\lfloor \frac{cj + c - b - 1}{a} \right\rfloor \\ &= nm - f(c, c - b - 1, a, m - 1) \end{aligned}$$

类欧几里德算法

- 所以可以推出:

$$\begin{aligned} f(a, b, c, n) &= \sum_{j=0}^{m-1} \sum_{i=0}^n \left[\left\lfloor \frac{cj + c - b - 1}{a} \right\rfloor < i \right] \\ &= \sum_{j=0}^{m-1} \sum_{i=1}^n \left[\left\lfloor \frac{cj + c - b - 1}{a} \right\rfloor < i \right] \\ &= \sum_{j=0}^{m-1} \left(n - \sum_{i=1}^n \left[i \leq \left\lfloor \frac{cj + c - b - 1}{a} \right\rfloor \right] \right) \\ &= nm - \sum_{j=0}^{m-1} \left\lfloor \frac{cj + c - b - 1}{a} \right\rfloor \\ &= nm - f(c, c - b - 1, a, m - 1) \end{aligned}$$

- 接下来来点引荐的。

类欧几里德算法变式

- 要解决的问题：给出非负整数 $a, b, c, n (c \neq 0)$ 求出

$$g(a, b, c, n) = \sum_{i=0}^n \lfloor \frac{ai+b}{c} \rfloor^2 \quad h(a, b, c, n) = \sum_{i=0}^n i \lfloor \frac{ai+b}{c} \rfloor$$

类欧几里德算法变式

- 要解决的问题：给出非负整数 $a, b, c, n (c \neq 0)$ 求出

$$g(a, b, c, n) = \sum_{i=0}^n \lfloor \frac{ai+b}{c} \rfloor^2 \quad h(a, b, c, n) = \sum_{i=0}^n i \lfloor \frac{ai+b}{c} \rfloor$$

- 还是先变为 $0 \leq a, b < c$ 的形式（建议自己推一下，这里直接写出结果）：

$$\begin{aligned} g(a, b, c, n) &= \lfloor \frac{a}{c} \rfloor^2 \frac{n(n+1)(2n+1)}{6} + \lfloor \frac{b}{c} \rfloor^2 (n+1) \\ &\quad + g(a \bmod c, b \bmod c, c, n) + \lfloor \frac{a}{c} \rfloor \lfloor \frac{b}{c} \rfloor n(n+1) \\ &\quad + 2 \lfloor \frac{a}{c} \rfloor h(a \bmod c, b \bmod c, c, n) + 2 \lfloor \frac{b}{c} \rfloor f(a \bmod c, b \bmod c, c, n) \\ h(a, b, c, n) &= \lfloor \frac{a}{c} \rfloor \frac{n(n+1)(2n+1)}{6} + \lfloor \frac{b}{c} \rfloor \frac{n(n+1)}{2} \\ &\quad + h(a \bmod c, b \bmod c, c, n) \end{aligned}$$

类欧几里德算法变式

- 同样的，设 $m = \lfloor \frac{an+b}{c} \rfloor$ 。

类欧几里德算法变式

- 同样的, 设 $m = \lfloor \frac{an+b}{c} \rfloor$ 。
- 若 $m = 0$ 则 $g(a, b, c, n) = h(a, b, c, n) = 0$ 。

类欧几里德算法变式

- 同样的, 设 $m = \lfloor \frac{an+b}{c} \rfloor$ 。
- 若 $m = 0$ 则 $g(a, b, c, n) = h(a, b, c, n) = 0$ 。
- 否则根据 $n^2 = \sum_{j=0}^{n-1} (j+1)^2 - j^2 = \sum_{j=0}^{n-1} 2j+1$ 有

$$\begin{aligned} g(a, b, c, n) &= \sum_{i=0}^n \lfloor \frac{ai+b}{c} \rfloor^2 = \sum_{i=0}^n \sum_{j=0}^{m-1} \left[j \leq \lfloor \frac{ai+b}{c} \rfloor - 1 \right] (2j+1) \\ &= \sum_{j=0}^{m-1} (2j+1) \sum_{i=0}^n \left[\lfloor \frac{cj+c-b-1}{a} \rfloor < i \right] \\ &= \sum_{j=0}^{m-1} (2j+1) \left(n - \lfloor \frac{cj+c-b-1}{a} \rfloor \right) \\ &= m^2 n - 2h(c, c-b-1, a, m-1) - f(c, c-b-1, a, m-1) \end{aligned}$$

类欧几里德算法变式

■ 对于 h 有

$$\begin{aligned}h(a, b, c, n) &= \sum_{i=0}^n i \lfloor \frac{ai+b}{c} \rfloor = \sum_{i=0}^n i \sum_{j=0}^{m-1} \left[j \leq \lfloor \frac{ai+b}{c} \rfloor - 1 \right] \\&= \sum_{j=0}^{m-1} \sum_{i=1}^n \left[\lfloor \frac{cj+c-b-1}{a} \rfloor < i \right] i = \sum_{j=0}^{m-1} \left(\sum_{i=1}^n i - \sum_{i=1}^{\lfloor \frac{cj+c-b-1}{a} \rfloor} i \right) \\&= m \frac{n(n+1)}{2} - \frac{1}{2} g(c, c-b-1, a, m-1) \\&\quad - \frac{1}{2} f(c, c-b-1, a, m-1)\end{aligned}$$

类欧几里德算法变式

- 对于 h 有

$$\begin{aligned}h(a, b, c, n) &= \sum_{i=0}^n i \lfloor \frac{ai+b}{c} \rfloor = \sum_{i=0}^n i \sum_{j=0}^{m-1} \left[j \leq \lfloor \frac{ai+b}{c} \rfloor - 1 \right] \\&= \sum_{j=0}^{m-1} \sum_{i=1}^n \left[\lfloor \frac{cj+c-b-1}{a} \rfloor < i \right] i = \sum_{j=0}^{m-1} \left(\sum_{i=1}^n i - \sum_{i=1}^{\lfloor \frac{cj+c-b-1}{a} \rfloor} i \right) \\&= m \frac{n(n+1)}{2} - \frac{1}{2} g(c, c-b-1, a, m-1) \\&\quad - \frac{1}{2} f(c, c-b-1, a, m-1)\end{aligned}$$

- 因此维护 f, g, h 的三元组即可，时间复杂度同欧几里德算法。

CF1098E Fedya the Potter

- 给定长度为 $n \leq 5 \times 10^4$ 的序列 $0 \leq a_1, a_2, \dots, a_n \leq 10^5$ 。
- 将 $\gcd(a_l, a_{l+1}, \dots, a_r) (1 \leq l \leq r \leq n)$ 从小到大排序后得到长度为 $n(n+1)/2$ 的序列 b 。
- 将 $\sum_{i=l}^r b_i (1 \leq l \leq r \leq n(n+1)/2)$ 从小到大排序后得到序列 c 。求 c 的中位数。
- 若 c 的下标从 1 到 k ，则 c 的中位数定义为其第 $\lfloor (k+1)/2 \rfloor$ 个元素。

CF1098E Fedya the Potter sol

- 首先那个 $\gcd$ 可以用枚举右端点维护，一个右端点所对应的 $\gcd$ 只有 $\log$ 种。

CF1098E Fedya the Potter sol

- 首先那个 gcd 可以用枚举右端点维护，一个右端点所对应的 gcd 只有 $\log$ 种。
- 于是以 $\mathcal{O}(n \log^2 a_i)$ 的复杂度处理出 cnt 数组表示 b 中有 cnt_i 个 i 。

CF1098E Fedya the Potter sol

- 首先那个 $\gcd$ 可以用枚举右端点维护，一个右端点所对应的 $\gcd$ 只有 $\log$ 种。
- 于是以 $\mathcal{O}(n \log^2 a_i)$ 的复杂度处理出 cnt 数组表示 b 中有 cnt_i 个 i 。
- 考虑二分 mid ，于是问题转化为求出 c 中比 mid 小的数的数量，也就是 b 中有几个区间和小于 mid 。

CF1098E Fedya the Potter sol

- 首先那个 $\gcd$ 可以用枚举右端点维护，一个右端点所对应的 $\gcd$ 只有 $\log$ 种。
- 于是以 $\mathcal{O}(n \log^2 a_i)$ 的复杂度处理出 cnt 数组表示 b 中有 cnt_i 个 i 。
- 考虑二分 mid ，于是问题转化为求出 c 中比 mid 小的数的数量，也就是 b 中有几个区间和小于 mid 。
- 设 l, r 为 b 中区间 $[x, y]$ 左右端点的值 b_x, b_y ，对 $l = r$ 的区间可以直接计算，对 $l < r$ 的区间分类讨论：

CF1098E Fedya the Potter sol

- 首先那个 $\gcd$ 可以用枚举右端点维护，一个右端点所对应的 $\gcd$ 只有 $\log$ 种。
- 于是以 $\mathcal{O}(n \log^2 a_i)$ 的复杂度处理出 cnt 数组表示 b 中有 cnt_i 个 i 。
- 考虑二分 mid ，于是问题转化为求出 c 中比 mid 小的数的数量，也就是 b 中有几个区间和小于 mid 。
- 设 l, r 为 b 中区间 $[x, y]$ 左右端点的值 b_x, b_y ，对 $l = r$ 的区间可以直接计算，对 $l < r$ 的区间分类讨论：
- 首先一个区间必须满足 $sum(l, r) = \sum_{b_l < b_i < b_r} b_i < mid$ 才能有贡献。

CF1098E Fedya the Potter sol

- 首先那个 $\gcd$ 可以用枚举右端点维护，一个右端点所对应的 $\gcd$ 只有 $\log$ 种。
- 于是以 $\mathcal{O}(n \log^2 a_i)$ 的复杂度处理出 cnt 数组表示 b 中有 cnt_i 个 i 。
- 考虑二分 mid ，于是问题转化为求出 c 中比 mid 小的数的数量，也就是 b 中有几个区间和小于 mid 。
- 设 l, r 为 b 中区间 $[x, y]$ 左右端点的值 b_x, b_y ，对 $l = r$ 的区间可以直接计算，对 $l < r$ 的区间分类讨论：
- 首先一个区间必须满足 $sum(l, r) = \sum_{b_l < b_i < b_r} b_i < mid$ 才能有贡献。
- 如果 $\sum_{l \leq i \leq r} icnt_i < mid$ 则可以直接 $cnt_l \times cnt_r$ ，这种区间的数量较多，可以枚举 l 然后双指针求出 r 的区间。

CF1098E Fedya the Potter sol

- 否则这个区间一定满足 $[l-1, r]$ 无贡献或 $[l, r+1]$ 无贡献, 因此数量只有 $\mathcal{O}(n)$ 个, 双指针的过程中可以顺带求出这些区间是哪些。

CF1098E Fedya the Potter sol

- 否则这个区间一定满足 $[l-1, r]$ 无贡献或 $[l, r+1]$ 无贡献, 因此数量只有 $\mathcal{O}(n)$ 个, 双指针的过程中可以顺带求出这些区间是哪些。
- 对于这些区间的贡献可以写成以下形式:

$$\sum_{i=1}^{cnt_l} \sum_{j=1}^{cnt_r} [li + sum(i, j) + rj < mid]$$

CF1098E Fedya the Potter sol

- 否则这个区间一定满足 $[l-1, r]$ 无贡献或 $[l, r+1]$ 无贡献, 因此数量只有 $\mathcal{O}(n)$ 个, 双指针的过程中可以顺带求出这些区间是哪些。
- 对于这些区间的贡献可以写成以下形式:

$$\sum_{i=1}^{cnt_l} \sum_{j=1}^{cnt_r} [li + sum(i, j) + rj < mid]$$

- 用字母代替常量, 再稍加改动, 变为:

$$\sum_{i=0}^A \sum_{j=0}^B [ai + bj \leq c]$$

CF1098E Fedya the Potter sol

- 然后记 $s = \min(A, \lfloor \frac{c}{a} \rfloor), t = \min(B, \lfloor \frac{c}{b} \rfloor)$, 有:

$$\begin{aligned} & \sum_{i=0}^A \sum_{j=0}^B [ai + bj \leq c] \\ &= \sum_{i=0}^s \sum_{j=0}^t [a(s-i) + bj \leq c] \\ &= (s+1)(t+1) - \sum_{i=0}^s \sum_{j=0}^t [ai + bj \leq as + bt - c - 1] \\ &= (s+1)(t+1) - \sum_{i=0}^{\infty} \sum_{j=0}^{\infty} [ai + bj \leq as + bt - c - 1] \end{aligned}$$

CF1098E Fedya the Potter sol

- 对于下面那个形式, 记 $n = \lfloor \frac{c}{a} \rfloor$, 有:

$$\begin{aligned} & \sum_{i=0}^{\infty} \sum_{j=0}^{\infty} [ai + bj \leq c] \\ &= \sum_{i=0}^n \sum_{j=0}^{\infty} [a(n-i) + bj \leq c] \\ &= \sum_{i=0}^n \sum_{j=0}^{\infty} [j + 1 \leq \frac{ai + c - an + b}{b}] \\ &= f(a, c - an + b, b, n) \end{aligned}$$

- 给定正整数 $n(n \leq 10^9), r(r \leq 10^4)$, 求:

$$\sum_{i=1}^n (-1)^{\lfloor i\sqrt{r} \rfloor}$$

P5172 Sum sol

- 首先特判 $\sqrt{r}$ 是整数的情况，否则 $\sqrt{r}$ 是无理数。

P5172 Sum sol

- 首先特判 $\sqrt{r}$ 是整数的情况，否则 $\sqrt{r}$ 是无理数。
- 一种方法是 $(-1)^{\lfloor x \rfloor} = 1 - 2\lfloor x \rfloor + 4\lfloor \frac{x}{2} \rfloor$ 所以原问题变为求：

$$f(k, n) = \sum_{i=1}^n \lfloor ki \rfloor$$

P5172 Sum sol

- 首先特判 $\sqrt{r}$ 是整数的情况，否则 $\sqrt{r}$ 是无理数。
- 一种方法是 $(-1)^{\lfloor x \rfloor} = 1 - 2\lfloor x \rfloor + 4\lfloor \frac{x}{2} \rfloor$ 所以原问题变为求：

$$f(k, n) = \sum_{i=1}^n \lfloor ki \rfloor$$

- 根据 k 是否大于等于 1 分类讨论，推式子可以发现

$$f(k, n) = n\lfloor (k - \lfloor k \rfloor)n \rfloor - f\left(\frac{1}{k - \lfloor k \rfloor}, \lfloor (k - \lfloor k \rfloor)n \rfloor\right) + \lfloor k \rfloor \frac{n(n+1)}{2}$$

P5172 Sum sol

- 首先特判 $\sqrt{r}$ 是整数的情况，否则 $\sqrt{r}$ 是无理数。
- 一种方法是 $(-1)^{\lfloor x \rfloor} = 1 - 2\lfloor x \rfloor + 4\lfloor \frac{x}{2} \rfloor$ 所以原问题变为求：

$$f(k, n) = \sum_{i=1}^n \lfloor ki \rfloor$$

- 根据 k 是否大于等于 1 分类讨论，推式子可以发现

$$f(k, n) = n\lfloor (k - \lfloor k \rfloor)n \rfloor - f\left(\frac{1}{k - \lfloor k \rfloor}, \lfloor (k - \lfloor k \rfloor)n \rfloor\right) + \lfloor k \rfloor \frac{n(n+1)}{2}$$

- 由于每次 $k \geq 1$ 时 $k - \lfloor k \rfloor \leq \frac{1}{2}k$ 所以每两层 n 至少除以 2，所以总复杂度 $\mathcal{O}(\log n)$ 。

P5172 Sum sol

- 首先特判 $\sqrt{r}$ 是整数的情况，否则 $\sqrt{r}$ 是无理数。
- 一种方法是 $(-1)^{\lfloor x \rfloor} = 1 - 2\lfloor x \rfloor + 4\lfloor \frac{x}{2} \rfloor$ 所以原问题变为求：

$$f(k, n) = \sum_{i=1}^n \lfloor ki \rfloor$$

- 根据 k 是否大于等于 1 分类讨论，推式子可以发现

$$f(k, n) = n\lfloor (k - \lfloor k \rfloor)n \rfloor - f\left(\frac{1}{k - \lfloor k \rfloor}, \lfloor (k - \lfloor k \rfloor)n \rfloor\right) + \lfloor k \rfloor \frac{n(n+1)}{2}$$

- 由于每次 $k \geq 1$ 时 $k - \lfloor k \rfloor \leq \frac{1}{2}k$ 所以每两层 n 至少除以 2，所以总复杂度 $\mathcal{O}(\log n)$ 。
- 对于 k 的操作，可以写成 $k = \frac{a\sqrt{r}+b}{c}$ 的形式来维护，注意取倒数时容易炸 long long，所以需要约分（实测除以 c 也行）。

P5172 Sum sol

- 关于约分后不会炸 long long 的证明:

P5172 Sum sol

- 关于约分后不会炸 long long 的证明:
- $\frac{a\sqrt{r}+b}{c}$ 的倒数为 $\frac{ac\sqrt{r}-bc}{a^2r-b^2}$, 分子是 c 的倍数显然。

P5172 Sum sol

- 关于约分后不会炸 long long 的证明:
- $\frac{a\sqrt{r}+b}{c}$ 的倒数为 $\frac{ac\sqrt{r}-bc}{a^2r-b^2}$, 分子是 c 的倍数显然。
- 那么假设第一层的数是 k , 那么设 $k - \lfloor k \rfloor = k - x = \frac{a\sqrt{r}+b}{c}$, 则第二层的数为 $k' = \frac{1}{k-x} = \frac{ac\sqrt{r}-bc}{a^2r-b^2}$ 。

P5172 Sum sol

- 关于约分后不会炸 long long 的证明:
- $\frac{a\sqrt{r}+b}{c}$ 的倒数为 $\frac{ac\sqrt{r}-bc}{a^2r-b^2}$, 分子是 c 的倍数显然。
- 那么假设第一层的数是 k , 那么设 $k - \lfloor k \rfloor = k - x = \frac{a\sqrt{r}+b}{c}$, 则第二层的数为 $k' = \frac{1}{k-x} = \frac{ac\sqrt{r}-bc}{a^2r-b^2}$ 。
- 设 $z = a^2r - b^2$, 记 $y = \lfloor k' \rfloor$ 可得 $k' - y = \frac{ac\sqrt{r}-(bc+yz)}{z}$, 所以 $k'' = \frac{1}{k'-y} = \frac{acz\sqrt{r}+(bc+yz)z}{a^2c^2r-b^2c^2-2bcyz-y^2z^2}$, 其中 $a^2c^2r-b^2c^2-2bcyz-y^2z^2=(c^2-2bcy-y^2z)z$ 所以一定可以对 z 约分。

P5172 Sum sol

- 关于约分后不会炸 long long 的证明:
- $\frac{a\sqrt{r}+b}{c}$ 的倒数为 $\frac{ac\sqrt{r}-bc}{a^2r-b^2}$, 分子是 c 的倍数显然。
- 那么假设第一层的数是 k , 那么设 $k - \lfloor k \rfloor = k - x = \frac{a\sqrt{r}+b}{c}$, 则第二层的数为 $k' = \frac{1}{k-x} = \frac{ac\sqrt{r}-bc}{a^2r-b^2}$ 。
- 设 $z = a^2r - b^2$, 记 $y = \lfloor k' \rfloor$ 可得 $k' - y = \frac{ac\sqrt{r}-(bc+yz)}{z}$, 所以 $k'' = \frac{1}{k'-y} = \frac{acz\sqrt{r}+(bc+yz)z}{a^2c^2r-b^2c^2-2bcyz-y^2z^2}$, 其中 $a^2c^2r-b^2c^2-2bcyz-y^2z^2 = (c^2-2bcy-y^2z)z$ 所以一定可以对 z 约分。
- 于是约分后第二层的 $\frac{a\sqrt{r}+b}{c}$ 的倒数表示为 $\frac{\frac{z}{c}\sqrt{r}-b}{\frac{z}{c}} = \frac{a\sqrt{r}-b}{c'}$ 。

P5172 Sum sol

- 关于约分后不会炸 long long 的证明:
- $\frac{a\sqrt{r}+b}{c}$ 的倒数为 $\frac{ac\sqrt{r}-bc}{a^2r-b^2}$, 分子是 c 的倍数显然。
- 那么假设第一层的数是 k , 那么设 $k - \lfloor k \rfloor = k - x = \frac{a\sqrt{r}+b}{c}$, 则第二层的数为 $k' = \frac{1}{k-x} = \frac{ac\sqrt{r}-bc}{a^2r-b^2}$ 。
- 设 $z = a^2r - b^2$, 记 $y = \lfloor k' \rfloor$ 可得 $k' - y = \frac{ac\sqrt{r}-(bc+yz)}{z}$, 所以 $k'' = \frac{1}{k'-y} = \frac{acz\sqrt{r}+(bc+yz)z}{a^2c^2r-b^2c^2-2bcyz-y^2z^2}$, 其中 $a^2c^2r-b^2c^2-2bcyz-y^2z^2=(c^2-2bcy-y^2z)z$ 所以一定可以对 z 约分。
- 于是约分后第二层的 $\frac{a\sqrt{r}+b}{c}$ 的倒数表示为 $\frac{\frac{a\sqrt{r}-b}{\frac{z}{c}}}{\frac{z}{c}} = \frac{a\sqrt{r}-b}{c'}$ 。
- 第三层 $k - t = \frac{a\sqrt{r}-b-c't}{c'}$ 的倒数的分母 $a^2r-b^2-2bc't-(c')^2t^2=(c-2bt-c't^2)c'$, 也可以对 c' 约分。

P5172 Sum sol

- 关于约分后不会炸 long long 的证明:
- $\frac{a\sqrt{r}+b}{c}$ 的倒数为 $\frac{ac\sqrt{r}-bc}{a^2r-b^2}$, 分子是 c 的倍数显然。
- 那么假设第一层的数是 k , 那么设 $k - \lfloor k \rfloor = k - x = \frac{a\sqrt{r}+b}{c}$, 则第二层的数为 $k' = \frac{1}{k-x} = \frac{ac\sqrt{r}-bc}{a^2r-b^2}$ 。
- 设 $z = a^2r - b^2$, 记 $y = \lfloor k' \rfloor$ 可得 $k' - y = \frac{ac\sqrt{r}-(bc+yz)}{z}$, 所以 $k'' = \frac{1}{k'-y} = \frac{acz\sqrt{r}+(bc+yz)z}{a^2c^2r-b^2c^2-2bcyz-y^2z^2}$, 其中 $a^2c^2r-b^2c^2-2bcyz-y^2z^2=(c^2-2bcy-y^2z)z$ 所以一定可以对 z 约分。
- 于是约分后第二层的 $\frac{a\sqrt{r}+b}{c}$ 的倒数表示为 $\frac{a\sqrt{r}-b}{\frac{z}{c}} = \frac{a\sqrt{r}-b}{c'}$ 。
- 第三层 $k - t = \frac{a\sqrt{r}-b-c't}{c'}$ 的倒数的分母 $a^2r-b^2-2bc't-(c')^2t^2=(c-2bt-c't^2)c'$, 也可以对 c' 约分。
- 整个过程中 a, k, c 非负, a 的大小从第二层后始终不变, $c = a^2r - b^2$ 不会超过 a^2r , $|b|$ 不会超过 $a\sqrt{r}$ 。

- 还有一种引荐（复杂度是一样的，常数更小）：设

$$f(k, n) = \sum_{i=1}^n (-1)^{\lfloor ik \rfloor}$$

- 还有一种引荐（复杂度是一样的，常数更小）：设

$$f(k, n) = \sum_{i=1}^n (-1)^{\lfloor ik \rfloor}$$

- 对于 $k \geq 2$ 的情况，显然 $f(k, n) = f(k-2, n)$ 。

- 还有一种引荐（复杂度是一样的，常数更小）：设

$$f(k, n) = \sum_{i=1}^n (-1)^{\lfloor ik \rfloor}$$

- 对于 $k \geq 2$ 的情况，显然 $f(k, n) = f(k - 2, n)$ 。
- 对于 $1 \leq k < 2$ 的情况，考虑
 $(-1)^{\lfloor ki \rfloor} = (-1)^{-\lceil (2-k)i \rceil} = (-1)^{-\lfloor (2-k)i \rfloor + 1}$ 所以
 $f(k, n) = -f(2 - k, n)$ 。

- 还有一种引荐（复杂度是一样的，常数更小）：设

$$f(k, n) = \sum_{i=1}^n (-1)^{\lfloor ik \rfloor}$$

- 对于 $k \geq 2$ 的情况，显然 $f(k, n) = f(k-2, n)$ 。
- 对于 $1 \leq k < 2$ 的情况，考虑
 $(-1)^{\lfloor ki \rfloor} = (-1)^{-\lceil (2-k)i \rceil} = (-1)^{-\lfloor (2-k)i \rfloor + 1}$ 所以
 $f(k, n) = -f(2-k, n)$ 。
- 对于 $0 \leq k < 1$ 的情况，设 $l = \frac{1}{k}$ ，则式子意义为：以长度为 l 的段交替黑白染色，求出 0 到 n 中的整点的黑色减白色的数量。

- 还有一种引荐（复杂度是一样的，常数更小）：设

$$f(k, n) = \sum_{i=1}^n (-1)^{\lfloor ik \rfloor}$$

- 对于 $k \geq 2$ 的情况，显然 $f(k, n) = f(k-2, n)$ 。
- 对于 $1 \leq k < 2$ 的情况，考虑
 $(-1)^{\lfloor ki \rfloor} = (-1)^{-\lceil (2-k)i \rceil} = (-1)^{-\lfloor (2-k)i \rfloor + 1}$ 所以
 $f(k, n) = -f(2-k, n)$ 。
- 对于 $0 \leq k < 1$ 的情况，设 $l = \frac{1}{k}$ ，则式子意义为：以长度为 l 的段交替黑白染色，求出 0 到 n 中的整点的黑色减白色的数量。
- 那么设 m 是最多的完整的一黑一白段的个数，最后的零散段直接计算对答案的贡献。对于前面的每段都至少有 $\lfloor l \rfloor$ 个点，这部分贡献相互抵消，所以可以将 l 对 1 取模，然后再加上 $f(\frac{1}{l}, \lfloor \frac{2m}{l} \rfloor)$ 。

万能欧几里德算法

- 来点养荐的。

万能欧几里德算法

- 来点养荐的。
- 要解决的问题：有两种操作 U, R 组成一个操作序列，其中共有 n 个 R ，第 $i (1 \leq i \leq n)$ 个 R 前有 $\lfloor \frac{ai+b}{c} \rfloor - \lfloor \frac{b}{c} \rfloor (0 \leq a, b, c-1)$ 个 U 操作，末尾是 R 操作，操作可以 $\mathcal{O}(T)$ 合并，求幂次。

万能欧几里德算法

- 来点养荐的。
- 要解决的问题：有两种操作 U, R 组成一个操作序列，其中共有 n 个 R ，第 i ($1 \leq i \leq n$) 个 R 前有 $\lfloor \frac{ai+b}{c} \rfloor - \lfloor \frac{b}{c} \rfloor$ ($0 \leq a, b, c-1$) 个 U 操作，末尾是 R 操作，操作可以 $\mathcal{O}(T)$ 合并，求幂次。
- 直观理解：考虑一条直线 $y = ax + b$ ，按 x 从 0 到 n 移动点 (x, y) ，每当 y 为正整数时执行 U 操作， x 为正整数时执行 R 操作，同时满足时先执行 U 操作。

万能欧几里德算法

- 来点养荐的。
- 要解决的问题：有两种操作 U, R 组成一个操作序列，其中共有 n 个 R ，第 $i (1 \leq i \leq n)$ 个 R 前有 $\lfloor \frac{ai+b}{c} \rfloor - \lfloor \frac{b}{c} \rfloor (0 \leq a, b, c-1)$ 个 U 操作，末尾是 R 操作，操作可以 $\mathcal{O}(T)$ 合并，求幂次。
- 直观理解：考虑一条直线 $y = ax + b$ ，按 x 从 0 到 n 移动点 (x, y) ，每当 y 为正整数时执行 U 操作， x 为正整数时执行 R 操作，同时满足时先执行 U 操作。
- 考虑当 $b \geq c$ 时可以 b 对 c 取模。当 $a \geq c$ 时每个 R 操作前都有至少 $k = \lfloor \frac{a}{c} \rfloor$ 个连续的 U 操作，所以可以取 $R' = U^k R$ ，然后将 a 对 c 取模。

万能欧几里德算法

- 来点养荐的。
- 要解决的问题：有两种操作 U, R 组成一个操作序列，其中共有 n 个 R ，第 $i (1 \leq i \leq n)$ 个 R 前有 $\lfloor \frac{ai+b}{c} \rfloor - \lfloor \frac{b}{c} \rfloor (0 \leq a, b, c-1)$ 个 U 操作，末尾是 R 操作，操作可以 $\mathcal{O}(T)$ 合并，求幂次。
- 直观理解：考虑一条直线 $y = ax + b$ ，按 x 从 0 到 n 移动点 (x, y) ，每当 y 为正整数时执行 U 操作， x 为正整数时执行 R 操作，同时满足时先执行 U 操作。
- 考虑当 $b \geq c$ 时可以 b 对 c 取模。当 $a \geq c$ 时每个 R 操作前都有至少 $k = \lfloor \frac{a}{c} \rfloor$ 个连续的 U 操作，所以可以取 $R' = U^k R$ ，然后将 a 对 c 取模。
- 接下来保证了 $0 \leq a, b < c$ ，考虑每个 U 前有几个 R 。

万能欧几里德算法

- 第 $1 \leq i \leq m = \lfloor \frac{an+b}{c} \rfloor$ 个 U 前的 R 的个数为:

$$\sum_{j=1}^n \left[\left\lfloor \frac{aj+b}{c} \right\rfloor < i \right] = \sum_{j=1}^n \left[j \leq \left\lfloor \frac{ci-b-1}{a} \right\rfloor \right] = \left\lfloor \frac{ci-b-1}{a} \right\rfloor$$

万能欧几里德算法

- 第 $1 \leq i \leq m = \lfloor \frac{an+b}{c} \rfloor$ 个 U 前的 R 的个数为:

$$\sum_{j=1}^n \left[\left\lfloor \frac{aj+b}{c} \right\rfloor < i \right] = \sum_{j=1}^n \left[j \leq \left\lfloor \frac{ci-b-1}{a} \right\rfloor \right] = \left\lfloor \frac{ci-b-1}{a} \right\rfloor$$

- 然而 $-b-1$ 不是非负数, 所以第一个 U 操作及前面的 R 需要单独计算, 然后第 $1 \leq i \leq m-1$ 个 U 前的 R 的个数变为 $\left\lfloor \frac{ci+c-b-1}{a} \right\rfloor - \left\lfloor \frac{c-b-1}{a} \right\rfloor$ 。

万能欧几里德算法

- 第 $1 \leq i \leq m = \lfloor \frac{an+b}{c} \rfloor$ 个 U 前的 R 的个数为:

$$\sum_{j=1}^n \left[\left\lfloor \frac{aj+b}{c} \right\rfloor < i \right] = \sum_{j=1}^n \left[j \leq \left\lfloor \frac{ci-b-1}{a} \right\rfloor \right] = \left\lfloor \frac{ci-b-1}{a} \right\rfloor$$

- 然而 $-b-1$ 不是非负数, 所以第一个 U 操作及前面的 R 需要单独计算, 然后第 $1 \leq i \leq m-1$ 个 U 前的 R 的个数变为 $\left\lfloor \frac{ci+c-b-1}{a} \right\rfloor - \left\lfloor \frac{c-b-1}{a} \right\rfloor$ 。
- 再处理一下最后 $n - \left\lfloor \frac{am+c-b-1}{c} \right\rfloor$ 个不在任何一个 U 前的 R , 就把问题转化为与原问题一致的形式了。时间复杂度 $\mathcal{O}(T \log)$ 。

万能欧几里德算法

- 第 $1 \leq i \leq m = \lfloor \frac{an+b}{c} \rfloor$ 个 U 前的 R 的个数为:

$$\sum_{j=1}^n \left[\left\lfloor \frac{aj+b}{c} \right\rfloor < i \right] = \sum_{j=1}^n \left[j \leq \left\lfloor \frac{ci-b-1}{a} \right\rfloor \right] = \left\lfloor \frac{ci-b-1}{a} \right\rfloor$$

- 然而 $-b-1$ 不是非负数, 所以第一个 U 操作及前面的 R 需要单独计算, 然后第 $1 \leq i \leq m-1$ 个 U 前的 R 的个数变为 $\left\lfloor \frac{ci+c-b-1}{a} \right\rfloor - \left\lfloor \frac{c-b-1}{a} \right\rfloor$ 。
- 再处理一下最后 $n - \left\lfloor \frac{am+c-b-1}{c} \right\rfloor$ 个不在任何一个 U 前的 R , 就把问题转化为与原问题一致的形式了。时间复杂度 $\mathcal{O}(T \log)$ 。
- 把上面的过程写成式子 (其中 $g(x) = \left\lfloor \frac{cx+c-b-1}{a} \right\rfloor$):

$$f(a, b, c, n, U, R) = \begin{cases} f(a \bmod c, b \bmod c, c, n, U, U^{\lfloor \frac{a}{c} \rfloor} R) & a \geq c \vee b \geq c \\ R^{g(1)} U f(c, c-b-1, a, m-1, R, U) R^{n-g(m-1)} & 0 \leq a, b < c \end{cases}$$

loj6440 万能欧几里得

■ 求

$$\sum_{i=1}^n A^i B^{\lfloor \frac{ai+b}{c} \rfloor}$$

- 其中 $0 \leq a, b, c, \lfloor \frac{an}{c} \rfloor \leq 10^{18}, c \neq 0$, A, B 是 $N \times N$ ($1 \leq N \leq 20$) 的矩阵, 输出答案矩阵的每个元素对 998244353 取模的结果。

loj6440 万能欧几里得 sol

- 维护矩阵的三元组 (U, V, S) 表示当前的 $A^i, B^{\lfloor \frac{ai+b}{c} \rfloor}, \Sigma$, 初始时 $U = I, V = B^{\lfloor \frac{b}{c} \rfloor}, S = O$ 。

loj6440 万能欧几里得 sol

- 维护矩阵的三元组 (U, V, S) 表示当前的 $A^i, B^{\lfloor \frac{ai+b}{c} \rfloor}, \Sigma$, 初始时 $U = I, V = B^{\lfloor \frac{b}{c} \rfloor}, S = O$ 。
- 定义单次操作（同样用矩阵的三元组表示） (X, Y, Z) 为 $(U, V, S) \longrightarrow (UX, YV, S + UZY)$, 发现它具有结合律。

loj6440 万能欧几里得 sol

- 维护矩阵的三元组 (U, V, S) 表示当前的 $A^i, B^{\lfloor \frac{ai+b}{c} \rfloor}, \Sigma$, 初始时 $U = I, V = B^{\lfloor \frac{b}{c} \rfloor}, S = O$ 。
- 定义单次操作（同样用矩阵的三元组表示） (X, Y, Z) 为 $(U, V, S) \rightarrow (UX, YV, S + UZY)$, 发现它具有结合律。
- 令 $U = (I, B, O), R = (A, I, A)$ 万欧即可。

目录

1 数论基础

2 欧几里德算法

3 积性函数

■ 例题

4 筛法

积性函数

- 数论函数：数论函数指定义域为正整数的函数。数论函数也可以视作一个数列。

积性函数

- 数论函数：数论函数指定义域为正整数的函数。数论函数也可以视作一个数列。
- 积性函数：若数论函数 $f(n)$ 满足 $f(1) = 1$ 且对于任意互素的正整数 x, y 都有 $f(xy) = f(x)f(y)$ ，则 $f(n)$ 为积性函数。

积性函数

- 数论函数：数论函数指定义域为正整数的函数。数论函数也可以视为一个数列。
- 积性函数：若数论函数 $f(n)$ 满足 $f(1) = 1$ 且对于任意互素的正整数 x, y 都有 $f(xy) = f(x)f(y)$ ，则 $f(n)$ 为积性函数。
- 完全积性函数：若数论函数 $f(n)$ 满足 $f(1) = 1$ 且对于任意正整数 x, y 都有 $f(xy) = f(x)f(y)$ ，则 $f(n)$ 为完全积性函数。

积性函数

- 数论函数：数论函数指定义域为正整数的函数。数论函数也可以视为一个数列。
- 积性函数：若数论函数 $f(n)$ 满足 $f(1) = 1$ 且对于任意互素的正整数 x, y 都有 $f(xy) = f(x)f(y)$ ，则 $f(n)$ 为积性函数。
- 完全积性函数：若数论函数 $f(n)$ 满足 $f(1) = 1$ 且对于任意正整数 x, y 都有 $f(xy) = f(x)f(y)$ ，则 $f(n)$ 为完全积性函数。
- 显然，积性函数的取值可以由所有素数幂次处的取值确定，完全积性函数可以由所有素数处的取值确定。

积性函数

- 数论函数：数论函数指定义域为正整数的函数。数论函数也可以视为一个数列。
- 积性函数：若数论函数 $f(n)$ 满足 $f(1) = 1$ 且对于任意互素的正整数 x, y 都有 $f(xy) = f(x)f(y)$ ，则 $f(n)$ 为积性函数。
- 完全积性函数：若数论函数 $f(n)$ 满足 $f(1) = 1$ 且对于任意正整数 x, y 都有 $f(xy) = f(x)f(y)$ ，则 $f(n)$ 为完全积性函数。
- 显然，积性函数的取值可以由所有素数幂次处的取值确定，完全积性函数可以由所有素数处的取值确定。
- 由定义可知，积性函数的乘积还是积性函数，完全积性函数乘积还是完全积性函数。

积性函数

- 以下是一些积性函数的例子：

积性函数

- 以下是一些积性函数的例子：
- 单位函数 $\epsilon(n) = [n = 1]$ 。

积性函数

- 以下是一些积性函数的例子:
- 单位函数 $\epsilon(n) = [n = 1]$ 。
- 恒等函数 $\text{id}_k(n) = n^k$, $\text{id}_1(n)$ 记作 $\text{id}(n)$ 。

积性函数

- 以下是一些积性函数的例子：
- 单位函数 $\epsilon(n) = [n = 1]$ 。
- 恒等函数 $\text{id}_k(n) = n^k$, $\text{id}_1(n)$ 记作 $\text{id}(n)$ 。
- 常数函数 $1(n) = 1$ 。（以上三项同时还是完全积性函数）

积性函数

- 以下是一些积性函数的例子：
- 单位函数 $\epsilon(n) = [n = 1]$ 。
- 恒等函数 $\text{id}_k(n) = n^k$, $\text{id}_1(n)$ 记作 $\text{id}(n)$ 。
- 常数函数 $1(n) = 1$ 。(以上三项同时还是完全积性函数)
- 除数函数 $\sigma_k(n) = \sum_{d|n} d^k$ 。 $\sigma_0(n)$ 记作 $d(n)$, $\sigma_1(n)$ 记作 $\sigma(n)$ 。

积性函数

- 以下是一些积性函数的例子：
- 单位函数 $\epsilon(n) = [n = 1]$ 。
- 恒等函数 $\text{id}_k(n) = n^k$, $\text{id}_1(n)$ 记作 $\text{id}(n)$ 。
- 常数函数 $1(n) = 1$ 。(以上三项同时还是完全积性函数)
- 除数函数 $\sigma_k(n) = \sum_{d|n} d^k$ 。 $\sigma_0(n)$ 记作 $d(n)$, $\sigma_1(n)$ 记作 $\sigma(n)$ 。
- 欧拉函数 $\varphi(n) = \sum_{i=1}^n [\gcd(i, n) == 1]$ 。

积性函数

- 以下是一些积性函数的例子:
- 单位函数 $\epsilon(n) = [n = 1]$ 。
- 恒等函数 $\text{id}_k(n) = n^k$, $\text{id}_1(n)$ 记作 $\text{id}(n)$ 。
- 常数函数 $1(n) = 1$ 。(以上三项同时还是完全积性函数)
- 除数函数 $\sigma_k(n) = \sum_{d|n} d^k$ 。 $\sigma_0(n)$ 记作 $d(n)$, $\sigma_1(n)$ 记作 $\sigma(n)$ 。
- 欧拉函数 $\varphi(n) = \sum_{i=1}^n [\text{gcd}(i, n) == 1]$ 。
- 莫比乌斯函数
$$\mu(n) = \begin{cases} 1 & n = 1 \\ 0 & \exists d > 1, d^2 \mid n, \text{ 其中 } \omega(n) \text{ 是 } n \\ (-1)^{\omega(n)} & \text{otherwise} \end{cases}$$

的不同质因子个数。

狄利克雷卷积

- 设 $f(n), g(n)$ 是两个数论函数则它们的狄利克雷卷积 $h(n)$ 也是一个数论函数，定义为：

$$h(n) = \sum_{d|n} f(d)g\left(\frac{n}{d}\right) = \sum_{ab=n} f(a)g(b)$$

记为 $h = f * g$ 。

狄利克雷卷积

- 设 $f(n), g(n)$ 是两个数论函数则它们的狄利克雷卷积 $h(n)$ 也是一个数论函数，定义为：

$$h(n) = \sum_{d|n} f(d)g\left(\frac{n}{d}\right) = \sum_{ab=n} f(a)g(b)$$

记为 $h = f * g$ 。

- 性质：

狄利克雷卷积

- 设 $f(n), g(n)$ 是两个数论函数则它们的狄利克雷卷积 $h(n)$ 也是一个数论函数，定义为：

$$h(n) = \sum_{d|n} f(d)g\left(\frac{n}{d}\right) = \sum_{ab=n} f(a)g(b)$$

记为 $h = f * g$ 。

- 性质：
- 交换律： $f * g = g * f$ 。

狄利克雷卷积

- 设 $f(n), g(n)$ 是两个数论函数则它们的狄利克雷卷积 $h(n)$ 也是一个数论函数，定义为：

$$h(n) = \sum_{d|n} f(d)g\left(\frac{n}{d}\right) = \sum_{ab=n} f(a)g(b)$$

记为 $h = f * g$ 。

- 性质：
- 交换律： $f * g = g * f$ 。
- 结合律： $(f * g) * h = f * (g * h)$ 。

狄利克雷卷积

- 设 $f(n), g(n)$ 是两个数论函数则它们的狄利克雷卷积 $h(n)$ 也是一个数论函数，定义为：

$$h(n) = \sum_{d|n} f(d)g\left(\frac{n}{d}\right) = \sum_{ab=n} f(a)g(b)$$

记为 $h = f * g$ 。

- 性质：
- 交换律： $f * g = g * f$ 。
- 结合律： $(f * g) * h = f * (g * h)$ 。
- 分配律： $(f + g) * h = f * h + g * h$ 。

狄利克雷卷积

- 设 $f(n), g(n)$ 是两个数论函数则它们的狄利克雷卷积 $h(n)$ 也是一个数论函数, 定义为:

$$h(n) = \sum_{d|n} f(d)g\left(\frac{n}{d}\right) = \sum_{ab=n} f(a)g(b)$$

记为 $h = f * g$ 。

- 性质:
- 交换律: $f * g = g * f$ 。
- 结合律: $(f * g) * h = f * (g * h)$ 。
- 分配律: $(f + g) * h = f * h + g * h$ 。
- 等式的性质: $f = g$ 的充要条件是 $f * h = g * h$, 其中 $h(1) \neq 0$ 。

狄利克雷卷积

- 单位元: $f * \epsilon = f$ 。

狄利克雷卷积

- 单位元: $f * \epsilon = f$ 。
- 逆元: 对于任何一个满足 $f(1) \neq 0$ 的数论函数, 如果如果有另一个数论函数 $g(n)$ 满足 $f * g = \epsilon$, 则称 $g(n)$ 是 $f(n)$ 的逆元, 由上面的性质可知逆元是唯一的, 而且可以构造出 $g(n)$ 的表达式:

$$g(n) = \frac{\epsilon(n) - \sum_{d|n, d \neq n} g(d)f(\frac{n}{d})}{f(1)}$$

狄利克雷卷积

- 单位元: $f * \epsilon = f$ 。
- 逆元: 对于任何一个满足 $f(1) \neq 0$ 的数论函数, 如果如果有另一个数论函数 $g(n)$ 满足 $f * g = \epsilon$, 则称 $g(n)$ 是 $f(n)$ 的逆元, 由上面的性质可知逆元是唯一的, 而且可以构造出 $g(n)$ 的表达式:

$$g(n) = \frac{\epsilon(n) - \sum_{d|n, d \neq n} g(d)f(\frac{n}{d})}{f(1)}$$

- 对积性函数封闭: 两个积性函数的狄利克雷卷积也是积性函数, 积性函数的逆元也是积性函数。

狄利克雷卷积

- 以下是一些积性函数狄利克雷卷积的例子:

$$\begin{aligned}\epsilon &= \mu * 1 \\ \sigma_k &= \text{id}_k * 1 \\ \varphi &= \mu * \text{id} \\ \epsilon &= (\mu \cdot t) * t\end{aligned}$$

其中 t 是完全积性函数。

狄利克雷生成函数

- 对于数论函数 $f(n)$ ，定义其狄利克雷生成函数（DGF）为：

$$F(x) = \sum_{i=1}^{+\infty} \frac{f(i)}{i^x}$$

狄利克雷生成函数

- 对于数论函数 $f(n)$ ，定义其狄利克雷生成函数（DGF）为：

$$F(x) = \sum_{i=1}^{+\infty} \frac{f(i)}{i^x}$$

- 如果 $f(n)$ 是积性函数，那么其 DGF 可以由质数幂处的取值表示：

$$F(x) = \prod_{p \in \mathbb{P}} (1 + f(p)p^{-x} + f(p^2)p^{-2x} + \cdots) = \prod_{p \in \mathbb{P}} F_p(p^{-x})$$

其中 F_p 是 p 的幂次处点值的 OGF。

狄利克雷生成函数

- 对于数论函数 $f(n)$ ，定义其狄利克雷生成函数（DGF）为：

$$F(x) = \sum_{i=1}^{+\infty} \frac{f(i)}{i^x}$$

- 如果 $f(n)$ 是积性函数，那么其 DGF 可以由质数幂处的取值表示：

$$F(x) = \prod_{p \in \mathbb{P}} (1 + f(p)p^{-x} + f(p^2)p^{-2x} + \cdots) = \prod_{p \in \mathbb{P}} F_p(p^{-x})$$

其中 F_p 是 p 的幂次处点值的 OGF。

- 可以发现，数论函数的卷积等价于对应的狄利克雷生成函数的乘法。

狄利克雷生成函数

- 一些积性函数所对应的 DGF :

狄利克雷生成函数

- 一些积性函数所对应的 DGF :
- $\epsilon(n)$ 对应 $E(x) = 1$ 。

狄利克雷生成函数

- 一些积性函数所对应的 DGF :
- $\epsilon(n)$ 对应 $E(x) = 1$ 。
- $1(n)$ 对应黎曼函数 $\zeta(x) = \sum_{i \geq 1} \frac{1}{i^x}$ 。

狄利克雷生成函数

- 一些积性函数所对应的 DGF :
- $\epsilon(n)$ 对应 $E(x) = 1$ 。
- $1(n)$ 对应黎曼函数 $\zeta(x) = \sum_{i \geq 1} \frac{1}{i^x}$ 。
- $\text{id}_k(n)$ 对应 $\zeta(x - k)$ 。

狄利克雷生成函数

- 一些积性函数所对应的 DGF :
- $\epsilon(n)$ 对应 $E(x) = 1$ 。
- $1(n)$ 对应黎曼函数 $\zeta(x) = \sum_{i \geq 1} \frac{1}{i^x}$ 。
- $\text{id}_k(n)$ 对应 $\zeta(x - k)$ 。
- $\sigma_k(n)$ 对应 $\zeta(x - k)\zeta(x)$ 。

狄利克雷生成函数

- 一些积性函数所对应的 DGF :
- $\epsilon(n)$ 对应 $E(x) = 1$ 。
- $1(n)$ 对应黎曼函数 $\zeta(x) = \sum_{i \geq 1} \frac{1}{i^x}$ 。
- $\text{id}_k(n)$ 对应 $\zeta(x - k)$ 。
- $\sigma_k(n)$ 对应 $\zeta(x - k)\zeta(x)$ 。
- $\varphi(n)$ 对应 $\frac{\zeta(x-1)}{\zeta(x)}$ 。

狄利克雷生成函数

- 一些积性函数所对应的 DGF :
- $\epsilon(n)$ 对应 $E(x) = 1$ 。
- $1(n)$ 对应黎曼函数 $\zeta(x) = \sum_{i \geq 1} \frac{1}{i^x}$ 。
- $\text{id}_k(n)$ 对应 $\zeta(x - k)$ 。
- $\sigma_k(n)$ 对应 $\zeta(x - k)\zeta(x)$ 。
- $\varphi(n)$ 对应 $\frac{\zeta(x-1)}{\zeta(x)}$ 。
- $\mu(n)$ 对应 $\frac{1}{\zeta(x)}$ 。

狄利克雷生成函数

- 一些积性函数所对应的 DGF :
- $\epsilon(n)$ 对应 $E(x) = 1$ 。
- $1(n)$ 对应黎曼函数 $\zeta(x) = \sum_{i \geq 1} \frac{1}{i^x}$ 。
- $\text{id}_k(n)$ 对应 $\zeta(x - k)$ 。
- $\sigma_k(n)$ 对应 $\zeta(x - k)\zeta(x)$ 。
- $\varphi(n)$ 对应 $\frac{\zeta(x-1)}{\zeta(x)}$ 。
- $\mu(n)$ 对应 $\frac{1}{\zeta(x)}$ 。
- 当杜教筛时卷上的函数不容易构造时, 不妨从狄利克雷生成函数的角度入手。

莫比乌斯反演

■ 形式一: $f(n) = \sum_{d|n} g(d) \iff g(n) = \sum_{d|n} \mu\left(\frac{n}{d}\right) f(d)$

莫比乌斯反演

- 形式一: $f(n) = \sum_{d|n} g(d) \iff g(n) = \sum_{d|n} \mu\left(\frac{n}{d}\right)f(d)$
- 容易看出, 其等价于 $f = g * 1 \iff g = f * \mu$, 这可以通过两边同卷 μ 得到。

莫比乌斯反演

- 形式一: $f(n) = \sum_{d|n} g(d) \iff g(n) = \sum_{d|n} \mu\left(\frac{n}{d}\right)f(d)$
- 容易看出, 其等价于 $f = g * 1 \iff g = f * \mu$, 这可以通过两边同卷 μ 得到。
- 形式二: $f(n) = \sum_{n|d} g(d) \iff g(n) = \sum_{n|d} \mu\left(\frac{d}{n}\right)f(d)$

莫比乌斯反演

- 形式一: $f(n) = \sum_{d|n} g(d) \iff g(n) = \sum_{d|n} \mu\left(\frac{n}{d}\right) f(d)$
- 容易看出, 其等价于 $f = g * 1 \iff g = f * \mu$, 这可以通过两边同卷 μ 得到。
- 形式二: $f(n) = \sum_{n|d} g(d) \iff g(n) = \sum_{n|d} \mu\left(\frac{d}{n}\right) f(d)$

■

$$\begin{aligned} \sum_{n|d} \mu\left(\frac{d}{n}\right) f(d) &= \sum_{k \geq 1} \mu(k) f(nk) = \sum_{k \geq 1} \mu(k) \sum_{nk|d} g(d) \\ &= \sum_{n|d} g(d) \sum_{k|\frac{d}{n}} \mu(k) = \sum_{n|d} g(d) \epsilon\left(\frac{d}{n}\right) = g(n) \end{aligned}$$

莫比乌斯反演

■ 形式三:

$$f(n) = \sum_{i=1}^n t(i)g(\lfloor \frac{n}{i} \rfloor) \iff g(n) = \sum_{i=1}^n \mu(i)t(i)f(\lfloor \frac{n}{i} \rfloor)$$

其中 t 是完全积性函数。

莫比乌斯反演

■ 形式三:

$$f(n) = \sum_{i=1}^n t(i)g(\lfloor \frac{n}{i} \rfloor) \iff g(n) = \sum_{i=1}^n \mu(i)t(i)f(\lfloor \frac{n}{i} \rfloor)$$

其中 t 是完全积性函数。

- 设数论函数 $f'(n) = f(n) - f(n-1)$, $g'(n) = g(n) - g(n-1)$ ($f(0), g(0)$ 视为 0), 那么上式左边可以改写为:

$$\sum_{d=1}^n f'(d) = \sum_{ij \leq n} t(i)g'(j) = \sum_{d=1}^n \sum_{ij=d} t(i)g'(j) = \sum_{d=1}^n (t * g')(d)$$

莫比乌斯反演

- 形式三:

$$f(n) = \sum_{i=1}^n t(i)g(\lfloor \frac{n}{i} \rfloor) \iff g(n) = \sum_{i=1}^n \mu(i)t(i)f(\lfloor \frac{n}{i} \rfloor)$$

其中 t 是完全积性函数。

- 设数论函数 $f'(n) = f(n) - f(n-1)$, $g'(n) = g(n) - g(n-1)$ ($f(0), g(0)$ 视为 0), 那么上式左边可以改写为:

$$\sum_{d=1}^n f'(d) = \sum_{ij \leq n} t(i)g'(j) = \sum_{d=1}^n \sum_{ij=d} t(i)g'(j) = \sum_{d=1}^n (t * g')(d)$$

- 也就是 $f' = t * g'$, 而它等同于 $g' = (\mu \cdot t) * f'$ 也就是右边。

P3704 [SDOI2017]数字表格

- $1 \leq T \leq 1000$ 次询问，每次给出 $1 \leq n, m \leq 10^6$ ，求：

$$\prod_{i=1}^n \prod_{j=1}^m f_{\gcd(i,j)}$$

- 其中 $f_0 = 0, f_1 = 1, f_i = f_{i-2} + f_{i-1}$ ，答案对 $10^9 + 7$ 取模。

P3704 [SDOI2017]数字表格 sol

- 定义 $(f \oplus g)(n) = \exp(\ln(f) * g) = \prod_{i|j} f(i)^{g(j)}$ 。

P3704 [SDOI2017]数字表格 sol

- 定义 $(f \oplus g)(n) = \exp(\ln(f) * g) = \prod_{i|j} f(i)^{g(j)}$ 。
- 有 $(f \oplus g) \oplus h = f \oplus (g * h)$ 。

P3704 [SDOI2017]数字表格 sol

- 定义 $(f \oplus g)(n) = \exp(\ln(f) * g) = \prod_{i|j} f(i)^{g(j)}$ 。
- 有 $(f \oplus g) \oplus h = f \oplus (g * h)$ 。
- 所以 $f = g \oplus 1 \iff g = f \oplus \mu$ 。

P3704 [SDOI2017]数字表格 sol

- 定义 $(f \oplus g)(n) = \exp(\ln(f) * g) = \prod_{i|j} f(i)^{g(j)}$ 。
- 有 $(f \oplus g) \oplus h = f \oplus (g * h)$ 。
- 所以 $f = g \oplus 1 \iff g = f \oplus \mu$ 。
- 可以这么理解（同时这也是求 g 的方法）： f 是 g 的高维前缀积
 $\iff g$ 是 f 的高维差分（商）。

P3704 [SDOI2017]数字表格 sol

- 定义 $(f \oplus g)(n) = \exp(\ln(f) * g) = \prod_{i|j} f(i)^{g(j)}$ 。
- 有 $(f \oplus g) \oplus h = f \oplus (g * h)$ 。
- 所以 $f = g \oplus 1 \iff g = f \oplus \mu$ 。
- 可以这么理解（同时这也是求 g 的方法）： f 是 g 的高维前缀积
 $\iff g$ 是 f 的高维差分（商）。

■

$$\prod_{i=1}^n \prod_{j=1}^m f_{\gcd(i,j)} = \prod_{i=1}^n \prod_{j=1}^m \prod_{d|i, d|j} g_d = \prod_{d=1}^{\min(n,m)} g_d^{\lfloor \frac{n}{d} \rfloor \lfloor \frac{m}{d} \rfloor}$$

P3704 [SDOI2017]数字表格 sol

- 定义 $(f \oplus g)(n) = \exp(\ln(f) * g) = \prod_{i|j} f(i)^{g(j)}$ 。
- 有 $(f \oplus g) \oplus h = f \oplus (g * h)$ 。
- 所以 $f = g \oplus 1 \iff g = f \oplus \mu$ 。
- 可以这么理解（同时这也是求 g 的方法）： f 是 g 的高维前缀积
 $\iff g$ 是 f 的高维差分（商）。

■

$$\prod_{i=1}^n \prod_{j=1}^m f_{\gcd(i,j)} = \prod_{i=1}^n \prod_{j=1}^m \prod_{d|i, d|j} g_d = \prod_{d=1}^{\min(n,m)} g_d^{\lfloor \frac{n}{d} \rfloor \lfloor \frac{m}{d} \rfloor}$$

- 整除分块即可 $\mathcal{O}(n \log \log n + T \sqrt{n} \log \text{mod})$ 。

P5518 [MtOI2019]幽灵乐团

- $1 \leq T \leq 70$ 次询问, 每次给出 3 个正整数 $1 \leq A, B, C \leq 10^5$, 对 $type \in \{0, 1, 2\}$ 求:

$$\prod_{i=1}^A \prod_{j=1}^B \prod_{k=1}^C \left(\frac{\text{lcm}(i, j)}{\text{gcd}(i, k)} \right)^{f_{type}(i, j, k)}$$

其中

$$\begin{aligned} f_0(i, j, k) &= 1 \\ f_1(i, j, k) &= i \times j \times k \\ f_2(i, j, k) &= \text{gcd}(i, j, k) \end{aligned}$$

P5518 [MtOI2019]幽灵乐团 sol

- 首先 $\frac{\text{lcm}(i,j)}{\text{gcd}(i,k)} = \frac{i \times j}{\text{gcd}(i,j) \text{gcd}(i,k)}$ 所以问题转化为

$$\prod_{i=1}^A \prod_{j=1}^B \prod_{k=1}^C i^{f_{\text{type}}(i,j,k)}$$

$$\prod_{i=1}^A \prod_{j=1}^B \prod_{k=1}^C \text{gcd}(i,j)^{f_{\text{type}}(i,j,k)}$$

P5518 [MtOI2019]幽灵乐团 sol

- 首先 $\frac{\text{lcm}(i,j)}{\text{gcd}(i,k)} = \frac{i \times j}{\text{gcd}(i,j) \text{gcd}(i,k)}$ 所以问题转化为

$$\prod_{i=1}^A \prod_{j=1}^B \prod_{k=1}^C i^{f_{\text{type}}(i,j,k)}$$

$$\prod_{i=1}^A \prod_{j=1}^B \prod_{k=1}^C \text{gcd}(i,j)^{f_{\text{type}}(i,j,k)}$$

- 发现 $\prod$ 不好看, 所以先转化为

$$\exp \sum_{i=1}^A \sum_{j=1}^B \sum_{k=1}^C \ln(i) f_{\text{type}}(i,j,k)$$

$$\exp \sum_{i=1}^A \sum_{j=1}^B \sum_{k=1}^C \ln(\text{gcd}(i,j)) f_{\text{type}}(i,j,k)$$

P5518 [MtOI2019]幽灵乐团 sol

■ 对于 $type = 0$:

$$\begin{aligned} & \exp \sum_{i=1}^A \sum_{j=1}^B \sum_{k=1}^C \ln(i) = \exp \sum_{i=1}^A \ln(i) BC = (A!)^{BC} \\ & \exp \sum_{i=1}^A \sum_{j=1}^B \sum_{k=1}^C \ln(\gcd(i, j)) \\ &= \exp \sum_{d=1}^{\min(A, B)} \ln(d) \sum_{d|i \leq A} \sum_{d|j \leq B} [\gcd(i, j) = d] C \\ &= \exp \sum_{d=1}^{\min(A, B)} \ln(d) \sum_{id \leq A} \sum_{jd \leq B} \sum_{T|i, T|j} \mu(T) C \\ &= \exp \sum_{T=1}^{\min(A, B)} \mu(T) \sum_{d=1}^{\min(A, B)} \ln(d) \lfloor \frac{A}{dT} \rfloor \lfloor \frac{B}{dT} \rfloor C \end{aligned}$$

P5518 [MtOI2019]幽灵乐团 sol



$$\begin{aligned} &= \exp \sum_{T=1}^{\min(A,B)} \sum_{d|T} \ln(d) \mu\left(\frac{T}{d}\right) \lfloor \frac{A}{T} \rfloor \lfloor \frac{B}{T} \rfloor C \\ &= \left(\prod_{T=1}^{\min(A,B)} \left(\prod_{d|T} d^{\mu(\frac{T}{d})} \right)^{\lfloor \frac{A}{T} \rfloor \lfloor \frac{B}{T} \rfloor} \right)^C \end{aligned}$$

P5518 [MtOI2019]幽灵乐团 sol

■

$$\begin{aligned} &= \exp \sum_{T=1}^{\min(A,B)} \sum_{d|T} \ln(d) \mu\left(\frac{T}{d}\right) \left\lfloor \frac{A}{T} \right\rfloor \left\lfloor \frac{B}{T} \right\rfloor C \\ &= \left(\prod_{T=1}^{\min(A,B)} \left(\prod_{d|T} d^{\mu(\frac{T}{d})} \right)^{\left\lfloor \frac{A}{T} \right\rfloor \left\lfloor \frac{B}{T} \right\rfloor} \right)^C \end{aligned}$$

■ 其中 $\prod_{d|T} d^{\mu(\frac{T}{d})}$ 可以用上一题的方法预处理，剩下的整除分块。

P5518 [MtOI2019]幽灵乐团 sol

- 对于 $type = 1$ 令 $S(x) = \frac{x(x+1)}{2}$:

$$\begin{aligned} & \exp \sum_{i=1}^A \sum_{j=1}^B \sum_{k=1}^C \ln(i)ijk \\ &= \exp \sum_{i=1}^A \ln(i)i \sum_{j=1}^B \sum_{k=1}^C jk \\ &= \left(\prod_{i=1}^A i^i \right)^{S(B)S(C)} \end{aligned}$$

P5518 [MtOI2019]幽灵乐团 sol



$$\begin{aligned}
 & \exp \sum_{i=1}^A \sum_{j=1}^B \sum_{k=1}^C \ln(\gcd(i, j)) i j k \\
 &= \exp \sum_{d=1}^{\min(A, B)} \ln(d) d^2 \sum_{i d \leq A} \sum_{j d \leq B} \sum_{T | i, T | j} \mu(T) i j S(C) \\
 &= \exp \sum_{T=1}^{\min(A, B)} \sum_{d=1}^{\min(A, B)} \ln(d) d^2 T^2 \mu(T) S(\lfloor \frac{A}{Td} \rfloor) S(\lfloor \frac{B}{Td} \rfloor) S(C) \\
 &= \exp \sum_{T=1}^{\min(A, B)} \sum_{d | T} \ln(d) \mu(\frac{T}{d}) T^2 S(\lfloor \frac{A}{T} \rfloor) S(\lfloor \frac{B}{T} \rfloor) S(C)
 \end{aligned}$$

P5518 [MtOI2019]幽灵乐团 sol

■ 最后得到

$$\left(\prod_{T=1}^{\min(A,B)} \left(\left(\prod_{d|T} d^{\mu(\frac{T}{d})} \right)^{T^2} \right)^{S(\lfloor \frac{A}{T} \rfloor) S(\lfloor \frac{B}{T} \rfloor)} \right)^{S(C)}$$

P5518 [MtOI2019]幽灵乐团 sol

■ 最后得到

$$\left(\prod_{T=1}^{\min(A,B)} \left(\left(\prod_{d|T} d^{\mu(\frac{T}{d})} \right)^{T^2} \right)^{S(\lfloor \frac{A}{T} \rfloor) S(\lfloor \frac{B}{T} \rfloor)} \right)^{S(C)}$$

■ 中间只跟 T 有关的部分也是预处理。

P5518 [MtOI2019]幽灵乐团 sol

■ 对于 $type = 2$:

$$\begin{aligned} & \exp \sum_{i=1}^A \sum_{j=1}^B \sum_{k=1}^C \ln\left(\frac{\text{lcm}(i, j)}{\text{gcd}(i, k)}\right) \text{gcd}(i, j, k) \\ &= \exp \sum_{d=1} \varphi(d) \sum_{i=1}^{\lfloor \frac{A}{d} \rfloor} \sum_{j=1}^{\lfloor \frac{B}{d} \rfloor} \sum_{k=1}^{\lfloor \frac{C}{d} \rfloor} \ln\left(\frac{\text{lcm}(id, jd)}{\text{gcd}(id, kd)}\right) \\ &= \exp \sum_{d=1} \varphi(d) \sum_{i=1}^{\lfloor \frac{A}{d} \rfloor} \sum_{j=1}^{\lfloor \frac{B}{d} \rfloor} \sum_{k=1}^{\lfloor \frac{C}{d} \rfloor} \ln\left(\frac{\text{lcm}(i, j)}{\text{gcd}(i, k)}\right) \\ &= \prod_{d=1}^{\min(A, B, C)} \text{ans}_0\left(\left\lfloor \frac{A}{d} \right\rfloor, \left\lfloor \frac{B}{d} \right\rfloor, \left\lfloor \frac{C}{d} \right\rfloor\right)^{\varphi(d)} \end{aligned}$$

P5518 [MtOI2019]幽灵乐团 sol

- 对于 $type = 2$:

$$\begin{aligned} & \exp \sum_{i=1}^A \sum_{j=1}^B \sum_{k=1}^C \ln\left(\frac{\text{lcm}(i, j)}{\text{gcd}(i, k)}\right) \text{gcd}(i, j, k) \\ &= \exp \sum_{d=1} \varphi(d) \sum_{i=1}^{\lfloor \frac{A}{d} \rfloor} \sum_{j=1}^{\lfloor \frac{B}{d} \rfloor} \sum_{k=1}^{\lfloor \frac{C}{d} \rfloor} \ln\left(\frac{\text{lcm}(id, jd)}{\text{gcd}(id, kd)}\right) \\ &= \exp \sum_{d=1} \varphi(d) \sum_{i=1}^{\lfloor \frac{A}{d} \rfloor} \sum_{j=1}^{\lfloor \frac{B}{d} \rfloor} \sum_{k=1}^{\lfloor \frac{C}{d} \rfloor} \ln\left(\frac{\text{lcm}(i, j)}{\text{gcd}(i, k)}\right) \\ &= \prod_{d=1}^{\min(A, B, C)} \text{ans}_0\left(\left\lfloor \frac{A}{d} \right\rfloor, \left\lfloor \frac{B}{d} \right\rfloor, \left\lfloor \frac{C}{d} \right\rfloor\right)^{\varphi(d)} \end{aligned}$$

- 两层数论分块的复杂度是 $T(n) = \sum_{i=1}^{\sqrt{n}} \frac{\sqrt{n}}{\sqrt{i}} = \mathcal{O}(n^{\frac{3}{4}})$, 再乘快速幂的 $\log$ 。总复杂度 $\mathcal{O}((n + Tn^{\frac{3}{4}}) \log)$ 。

目录

1 数论基础

2 欧几里德算法

3 积性函数

4 筛法

线性筛

- 要求：质数幂次的值可以快速计算，函数满足积性。

线性筛

- 要求：质数幂次的值可以快速计算，函数满足积性。
- 过程：算出所有质数幂次的答案，然后合并。

线性筛

- 要求：质数幂次的值可以快速计算，函数满足积性。
- 过程：算出所有质数幂次的答案，然后合并。
- 总复杂度 $\mathcal{O}(n)$ 。

线性筛

- 要求：质数幂次的值可以快速计算，函数满足积性。
- 过程：算出所有质数幂次的答案，然后合并。
- 总复杂度 $\mathcal{O}(n)$ 。
- 优点：可以求出 1 到 n 处的前缀和。

杜教筛

- 要求: 数论函数 f 满足 $f * g = h$, g 和 h 在所有 $\lfloor \frac{n}{i} \rfloor$ 处的前缀和可以快速计算。

杜教筛

- 要求: 数论函数 f 满足 $f * g = h$, g 和 h 在所有 $\lfloor \frac{n}{i} \rfloor$ 处的前缀和可以快速计算。
- 过程: 记 f, g, h 的前缀和为 F, G, H , 那么 $H(n) = \sum_{i=1}^n h(i) = \sum_{ij \leq n} f(j)g(i) = F(n)g(1) + \sum_{i=2}^n g(i)F(\lfloor \frac{n}{i} \rfloor)$ 。

杜教筛

- 要求: 数论函数 f 满足 $f * g = h$, g 和 h 在所有 $\lfloor \frac{n}{i} \rfloor$ 处的前缀和可以快速计算。
- 过程: 记 f, g, h 的前缀和为 F, G, H , 那么 $H(n) = \sum_{i=1}^n h(i) = \sum_{ij \leq n} f(j)g(i) = F(n)g(1) + \sum_{i=2}^n g(i)F(\lfloor \frac{n}{i} \rfloor)$ 。
- 所以 $F(n) = \frac{H(n) - \sum_{i=2}^n g(i)F(\lfloor \frac{n}{i} \rfloor)}{g(1)}$ 。

杜教筛

- 要求: 数论函数 f 满足 $f * g = h$, g 和 h 在所有 $\lfloor \frac{n}{i} \rfloor$ 处的前缀和可以快速计算。
- 过程: 记 f, g, h 的前缀和为 F, G, H , 那么 $H(n) = \sum_{i=1}^n h(i) = \sum_{ij \leq n} f(j)g(i) = F(n)g(1) + \sum_{i=2}^n g(i)F(\lfloor \frac{n}{i} \rfloor)$ 。
- 所以 $F(n) = \frac{H(n) - \sum_{i=2}^n g(i)F(\lfloor \frac{n}{i} \rfloor)}{g(1)}$ 。
- 直接做复杂度是 $\mathcal{O}(n^{\frac{3}{4}})$ 的, 如果 f 满足积性, 则可以用线性筛预处理前 $n^{\frac{2}{3}}$ 的值, 从而优化到 $\mathcal{O}(n^{\frac{2}{3}})$ 。

杜教筛

- 要求: 数论函数 f 满足 $f * g = h$, g 和 h 在所有 $\lfloor \frac{n}{i} \rfloor$ 处的前缀和可以快速计算。
- 过程: 记 f, g, h 的前缀和为 F, G, H , 那么 $H(n) = \sum_{i=1}^n h(i) = \sum_{ij \leq n} f(j)g(i) = F(n)g(1) + \sum_{i=2}^n g(i)F(\lfloor \frac{n}{i} \rfloor)$ 。
- 所以 $F(n) = \frac{H(n) - \sum_{i=2}^n g(i)F(\lfloor \frac{n}{i} \rfloor)}{g(1)}$ 。
- 直接做复杂度是 $\mathcal{O}(n^{\frac{3}{4}})$ 的, 如果 f 满足积性, 则可以用线性筛预处理前 $n^{\frac{2}{3}}$ 的值, 从而优化到 $\mathcal{O}(n^{\frac{2}{3}})$ 。
- 优点: 快且可以求出所有 $\lfloor \frac{n}{i} \rfloor$ 处的前缀和。所以杜教筛套杜教筛, 整除分块套杜教筛的复杂度是对的。

杜教筛

- 要求: 数论函数 f 满足 $f * g = h$, g 和 h 在所有 $\lfloor \frac{n}{i} \rfloor$ 处的前缀和可以快速计算。
- 过程: 记 f, g, h 的前缀和为 F, G, H , 那么 $H(n) = \sum_{i=1}^n h(i) = \sum_{ij \leq n} f(j)g(i) = F(n)g(1) + \sum_{i=2}^n g(i)F(\lfloor \frac{n}{i} \rfloor)$ 。
- 所以 $F(n) = \frac{H(n) - \sum_{i=2}^n g(i)F(\lfloor \frac{n}{i} \rfloor)}{g(1)}$ 。
- 直接做复杂度是 $\mathcal{O}(n^{\frac{3}{4}})$ 的, 如果 f 满足积性, 则可以用线性筛预处理前 $n^{\frac{2}{3}}$ 的值, 从而优化到 $\mathcal{O}(n^{\frac{2}{3}})$ 。
- 优点: 快且可以求出所有 $\lfloor \frac{n}{i} \rfloor$ 处的前缀和。所以杜教筛套杜教筛, 整除分块套杜教筛的复杂度是对的。
- 然而这个 g 和 h 不是那么好找的。

Powerful Number 筛

- 要求: 函数 f 满足积性, 存在一个积性函数 g 使得 $\forall p \in \mathbb{P}, f(p) = g(p)$ 且 g 在所有 $\lfloor \frac{n}{i} \rfloor$ 处的前缀和可以快速计算, f 质数幂次的值可以快速计算。

Powerful Number 筛

- 要求: 函数 f 满足积性, 存在一个积性函数 g 使得
 $\forall p \in \mathbb{P}, f(p) = g(p)$ 且 g 在所有 $\lfloor \frac{n}{i} \rfloor$ 处的前缀和可以快速计算, f 质数幂次的值可以快速计算。
- 过程: 考虑积性函数 h 满足 $f = g * h$, 那么
 $\forall p \in \mathbb{P}, f(p) = g(1)h(p) + h(1)g(p) = h(p) + g(p)$ 而 $f(p) = g(p)$,
所以 $h(p) = 0$ 。

Powerful Number 筛

- 要求: 函数 f 满足积性, 存在一个积性函数 g 使得
 $\forall p \in \mathbb{P}, f(p) = g(p)$ 且 g 在所有 $\lfloor \frac{n}{i} \rfloor$ 处的前缀和可以快速计算, f 质数幂次的值可以快速计算。
- 过程: 考虑积性函数 h 满足 $f = g * h$, 那么
 $\forall p \in \mathbb{P}, f(p) = g(1)h(p) + h(1)g(p) = h(p) + g(p)$ 而 $f(p) = g(p)$, 所以 $h(p) = 0$ 。
- 由于 h 是积性函数, 所以 $h(n)$ 的值非 0 仅当 $n = \prod_{i=1}^s p_i^{k_i}$ 中
 $\forall 1 \leq i \leq s, k_i > 1$ 。这样的 n 称为 Powerful Number。

Powerful Number 筛

- 要求: 函数 f 满足积性, 存在一个积性函数 g 使得 $\forall p \in \mathbb{P}, f(p) = g(p)$ 且 g 在所有 $\lfloor \frac{n}{i} \rfloor$ 处的前缀和可以快速计算, f 质数幂次的值可以快速计算。
- 过程: 考虑积性函数 h 满足 $f = g * h$, 那么 $\forall p \in \mathbb{P}, f(p) = g(1)h(p) + h(1)g(p) = h(p) + g(p)$ 而 $f(p) = g(p)$, 所以 $h(p) = 0$ 。
- 由于 h 是积性函数, 所以 $h(n)$ 的值非 0 仅当 $n = \prod_{i=1}^s p_i^{k_i}$ 中 $\forall 1 \leq i \leq s, k_i > 1$ 。这样的 n 称为 Powerful Number。
- 考虑对于大于 1 的数 k 一定存在非负整数 x, y 使得 $2x + 3y = k$, 所以 $n = a^2 b^3$, 可以发现这样的数不超过 $\sum_{a=1}^{\sqrt{n}} \frac{\sqrt[3]{n}}{\sqrt[3]{a}} = \mathcal{O}(\sqrt{n})$ 个。

Powerful Number 筛

- 要求: 函数 f 满足积性, 存在一个积性函数 g 使得 $\forall p \in \mathbb{P}, f(p) = g(p)$ 且 g 在所有 $\lfloor \frac{n}{i} \rfloor$ 处的前缀和可以快速计算, f 质数幂次的值可以快速计算。
- 过程: 考虑积性函数 h 满足 $f = g * h$, 那么 $\forall p \in \mathbb{P}, f(p) = g(1)h(p) + h(1)g(p) = h(p) + g(p)$ 而 $f(p) = g(p)$, 所以 $h(p) = 0$ 。
- 由于 h 是积性函数, 所以 $h(n)$ 的值非 0 仅当 $n = \prod_{i=1}^s p_i^{k_i}$ 中 $\forall 1 \leq i \leq s, k_i > 1$ 。这样的 n 称为 Powerful Number。
- 考虑对于大于 1 的数 k 一定存在非负整数 x, y 使得 $2x + 3y = k$, 所以 $n = a^2 b^3$, 可以发现这样的数不超过 $\sum_{a=1}^{\sqrt{n}} \frac{\sqrt[3]{n}}{\sqrt[3]{i}} = \mathcal{O}(\sqrt{n})$ 个。
- 那么 $F(n) = \sum_{ij \leq n} h(i)g(j) = \sum_{1 \leq i \leq n \wedge i \in PN} h(i)G(\lfloor \frac{n}{i} \rfloor)$ 。

Powerful Number 筛

- 要求: 函数 f 满足积性, 存在一个积性函数 g 使得 $\forall p \in \mathbb{P}, f(p) = g(p)$ 且 g 在所有 $\lfloor \frac{n}{i} \rfloor$ 处的前缀和可以快速计算, f 质数幂次的值可以快速计算。
- 过程: 考虑积性函数 h 满足 $f = g * h$, 那么 $\forall p \in \mathbb{P}, f(p) = g(1)h(p) + h(1)g(p) = h(p) + g(p)$ 而 $f(p) = g(p)$, 所以 $h(p) = 0$ 。
- 由于 h 是积性函数, 所以 $h(n)$ 的值非 0 仅当 $n = \prod_{i=1}^s p_i^{k_i}$ 中 $\forall 1 \leq i \leq s, k_i > 1$ 。这样的 n 称为 Powerful Number。
- 考虑对于大于 1 的数 k 一定存在非负整数 x, y 使得 $2x + 3y = k$, 所以 $n = a^2 b^3$, 可以发现这样的数不超过 $\sum_{a=1}^{\sqrt{n}} \frac{\sqrt[3]{n}}{\sqrt[3]{i}} = \mathcal{O}(\sqrt{n})$ 个。
- 那么 $F(n) = \sum_{ij \leq n} h(i)g(j) = \sum_{1 \leq i \leq n \wedge i \in PN} h(i)G(\lfloor \frac{n}{i} \rfloor)$ 。
- 总复杂度 $\mathcal{O}(\sqrt{n} \log n + \sqrt{n} + T(n))$, 其中前面是算 $h(p^c)$ 的复杂度, $T(n)$ 是算 G 的复杂度。

min_25 筛

- 要求：函数 f 满足积性， $f(p)$ 是一个关于 p 的项数较少的多项式或其对应的完全积性函数可以快速求值，质数幂次的值可以快速计算。

min_25 筛

- 要求：函数 f 满足积性， $f(p)$ 是一个关于 p 的项数较少的多项式或其对应的完全积性函数可以快速求值，质数幂次的值可以快速计算。
- 过程：先算质数的答案再计算总的答案，如果已经算出了质数的答案，考虑记：

min_25 筛

- 要求：函数 f 满足积性， $f(p)$ 是一个关于 p 的项数较少的多项式或其对应的完全积性函数可以快速求值，质数幂次的值可以快速计算。
- 过程：先算质数的答案再计算总的答案，如果已经算出了质数的答案，考虑记：
- p_k 表示第 k 的素数。 $rk(i)$ 表示小于等于 i 的素数个数。

min_25 筛

- 要求: 函数 f 满足积性, $f(p)$ 是一个关于 p 的项数较少的多项式或其对应的完全积性函数可以快速求值, 质数幂次的值可以快速计算。
- 过程: 先算质数的答案再计算总的答案, 如果已经算出了质数的答案, 考虑记:
- p_k 表示第 k 的素数。 $rk(i)$ 表示小于等于 i 的素数个数。
- $lpf(i)$ 表示 i 的最小质因数, $lpf(1) = 1$ 。

min_25 筛

- 要求: 函数 f 满足积性, $f(p)$ 是一个关于 p 的项数较少的多项式或其对应的完全积性函数可以快速求值, 质数幂次的值可以快速计算。
- 过程: 先算质数的答案再计算总的答案, 如果已经算出了质数的答案, 考虑记:
- p_k 表示第 k 的素数。 $rk(i)$ 表示小于等于 i 的素数个数。
- $lpf(i)$ 表示 i 的最小质因数, $lpf(1) = 1$ 。
- $F_{prime}(n) = \sum_{i=1}^{rk(n)} f(p_i)$

min_25 筛

- 要求: 函数 f 满足积性, $f(p)$ 是一个关于 p 的项数较少的多项式或其对应的完全积性函数可以快速求值, 质数幂次的值可以快速计算。
- 过程: 先算质数的答案再计算总的答案, 如果已经算出了质数的答案, 考虑记:
- p_k 表示第 k 的素数。 $rk(i)$ 表示小于等于 i 的素数个数。
- $lpf(i)$ 表示 i 的最小质因数, $lpf(1) = 1$ 。
- $F_{prime}(n) = \sum_{i=1}^{rk(n)} f(p_i)$
- $F_k(n) = \sum_{i=2}^n [lpf(i) \geq p_k] f(i)$

min_25 筛

- 枚举合数的最小质因数可以推出:

$$\begin{aligned} F_k(n) &= F_{prime}(n) - F_{prime}(p_{k-1}) + \\ &\sum_{i=k}^{rk(\sqrt{n})} \sum_{c \geq 1 \wedge p_i^c \leq n} f(p_i^c) F_{i+1}(\lfloor \frac{n}{p_i^c} \rfloor) + [c > 1] f(p_i^c) \\ &= F_{prime}(n) - F_{prime}(p_{k-1}) + \\ &\sum_{i=k}^{rk(\sqrt{n})} \sum_{c \geq 1 \wedge p_i^{c+1} \leq n} f(p_i^c) F_{i+1}(\lfloor \frac{n}{p_i^c} \rfloor) + f(p_i^{c+1}) \end{aligned}$$

min_25 筛

- 枚举合数的最小质因数可以推出:

$$\begin{aligned} F_k(n) &= F_{prime}(n) - F_{prime}(p_{k-1}) + \\ &\sum_{i=k}^{rk(\sqrt{n})} \sum_{c \geq 1 \wedge p_i^c \leq n} f(p_i^c) F_{i+1}(\lfloor \frac{n}{p_i^c} \rfloor) + [c > 1] f(p_i^c) \\ &= F_{prime}(n) - F_{prime}(p_{k-1}) + \\ &\sum_{i=k}^{rk(\sqrt{n})} \sum_{c \geq 1 \wedge p_i^{c+1} \leq n} f(p_i^c) F_{i+1}(\lfloor \frac{n}{p_i^c} \rfloor) + f(p_i^{c+1}) \end{aligned}$$

- 这里如果不进行后缀和优化转移的话是 $\mathcal{O}(n^{1-\epsilon})$, 使用则是 $\mathcal{O}(\frac{n^{\frac{3}{4}}}{\log n})$ 的。

min_25 筛

- 把 $f(p)$ 拆开后问题转化为对应的 g 是完全积性函数的情况。

min_25 筛

- 把 $f(p)$ 拆开后问题转化为对应的 g 是完全积性函数的情况。
- 对于 $F_{prime}(n)$ 只有 $\lfloor \frac{n}{i} \rfloor$ 处的值需要使用, 设
 $G_k(n) = \sum_{i=2}^n [\text{lpf}(i) > p_k \vee i \in \mathbb{P}] g(i) (k \leq rk(\sqrt{n}))$ 可以推出

$$\begin{aligned} G_k(n) &= \sum_{i=2}^n [\text{lpf}(i) > p_k \vee i \in \mathbb{P}] g(i) \\ &= G_{k-1}(n) - g(p_k) (G_{k-1}(\lfloor \frac{n}{p_k} \rfloor) - G_{rk(\sqrt{p_k-1})}(p_{k-1})) \end{aligned}$$

min_25 筛

- 把 $f(p)$ 拆开后问题转化为对应的 g 是完全积性函数的情况。
- 对于 $F_{prime}(n)$ 只有 $\lfloor \frac{n}{i} \rfloor$ 处的值需要使用, 设
 $G_k(n) = \sum_{i=2}^n [\text{lpf}(i) > p_k \vee i \in \mathbb{P}] g(i) (k \leq rk(\sqrt{n}))$ 可以推出

$$\begin{aligned} G_k(n) &= \sum_{i=2}^n [\text{lpf}(i) > p_k \vee i \in \mathbb{P}] g(i) \\ &= G_{k-1}(n) - g(p_k) (G_{k-1}(\lfloor \frac{n}{p_k} \rfloor) - G_{rk(\sqrt{p_k-1})}(p_{k-1})) \end{aligned}$$

- 而 $G_0(n) = \sum_{i=2}^n g(i)$ 需要快速计算, 可以得到
 $F_{prime}(n) = G_{rk(\sqrt{n})}(n)$ 。总复杂度 $O(\frac{n^{\frac{3}{4}}}{\log n})$ 。

min_25 筛

- 把 $f(p)$ 拆开后问题转化为对应的 g 是完全积性函数的情况。
- 对于 $F_{prime}(n)$ 只有 $\lfloor \frac{n}{i} \rfloor$ 处的值需要使用, 设
 $G_k(n) = \sum_{i=2}^n [\text{lpf}(i) > p_k \vee i \in \mathbb{P}] g(i) (k \leq rk(\sqrt{n}))$ 可以推出

$$\begin{aligned} G_k(n) &= \sum_{i=2}^n [\text{lpf}(i) > p_k \vee i \in \mathbb{P}] g(i) \\ &= G_{k-1}(n) - g(p_k) (G_{k-1}(\lfloor \frac{n}{p_k} \rfloor) - G_{rk(\sqrt{p_k-1})}(p_{k-1})) \end{aligned}$$

- 而 $G_0(n) = \sum_{i=2}^n g(i)$ 需要快速计算, 可以得到

$$F_{prime}(n) = G_{rk(\sqrt{n})}(n)。总复杂度 O(\frac{n^{\frac{3}{4}}}{\log n})。$$

- 优点: 可以筛大部分积性函数, 且可以求质数处的点值和。

GL and HF